Computer Organization and Architecture

Dr. V S M Srinivasavarma

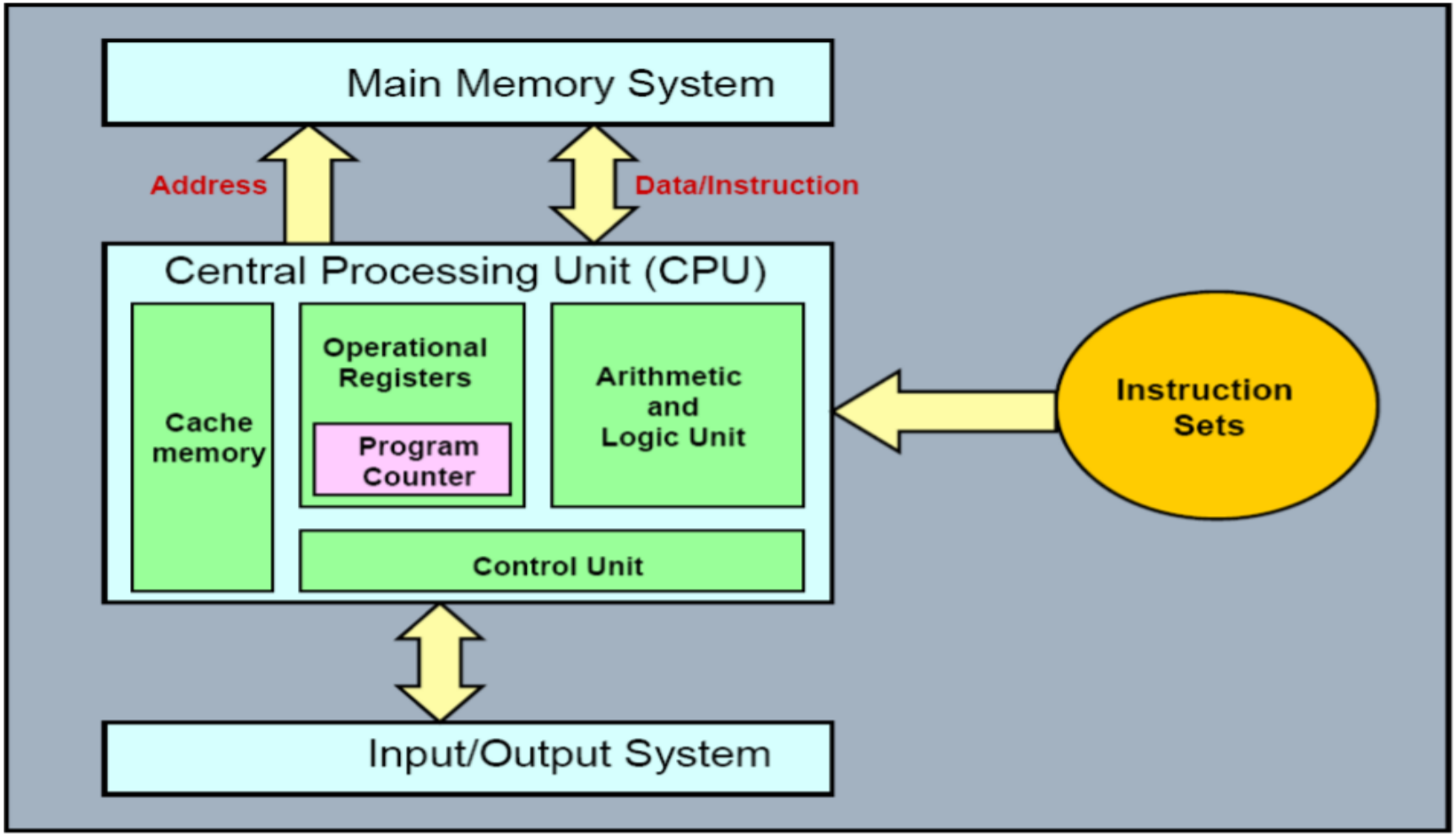
Syllabus

- Module-1: Function and Structure of a Computer, functional components of computer, Operational Concepts, Bus Structures, Memory Location and Addresses, Memory Operations, Instructions and Instruction Sequencing, Addressing modes, Instruction Encoding Format, RISC and CISC, Processor Performance Evaluation, Introduction to Microprocessor, Introduction to Assembly Language.

FUNCTIONAL UNITS OF COMPUTER

- **Input Unit**
- **Output Unit**
- **Central processing Unit (ALU and Control Units)**
- **Memory**
- **Bus Structure**

Basic Functional Unit of a Computer



Computer.?

- A computer is a sophisticated electronic calculating machine that:
  - Accepts input information,
  - Processes the information according to a list of internally stored instructions and
  - Produces the resulting output information.
- Functions performed by a computer are:
  - Accepting information to be processed as input.
  - Storing a list of instructions to process the information.
  - Processing the information according to the list of instructions.
  - Providing the results of the processing as output.

- **ALL computer functions are:**
  - **Data PROCESSING**
  - **Data STORAGE**
  - **Data MOVEMENT**
  - **CONTROL** → Coordinates How Information is Used

Data = Information

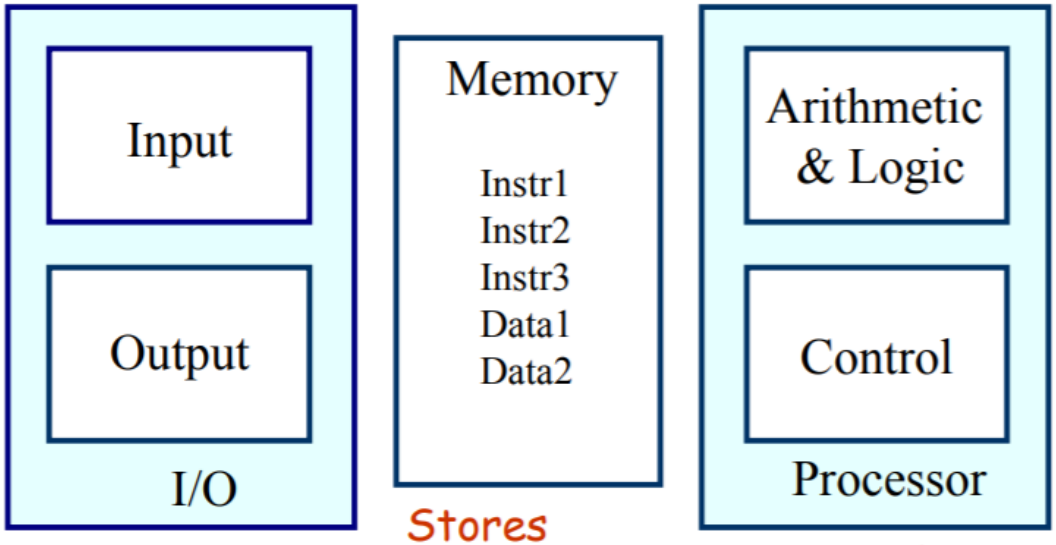
FUNCTIONS and Functional Units

The operations performed by a computer using the functional units can be summarized as follows:

- It accepts information (program and data) through input unit and transfers it to the memory.
- Information stored in the memory is fetched, under program control, into an arithmetic and logic unit for processing.
- Processed information leaves the computer through an output unit.
- The control unit controls all activities taking place inside a computer.

Input unit accepts information:
- Human operators,
- Electromechanical devices (keyboard)
- Other computers

Arithmetic and logic unit(ALU):
- Performs the desired operations on the input information as determined by instructions in the memory



Output unit sends results of processing:
- To a monitor display,
- To a printer

Stores information:
- Instructions,
- Data

Control unit coordinates various actions
- Input,
- Output
- Processing

Information in a computer -- *Instructions*

- Instructions specify commands to:
  - Transfer information within a computer (e.g., from memory to ALU)
  - Transfer of information between the computer and I/O devices (e.g., from keyboard to computer, or computer to printer)
  - Perform arithmetic and logic operations (e.g., Add two numbers, Perform a logical AND).
- A sequence of instructions to perform a task is called a program, which is stored in the memory.
- Processor fetches instructions that make up a program from the memory and performs the operations stated in those instructions.

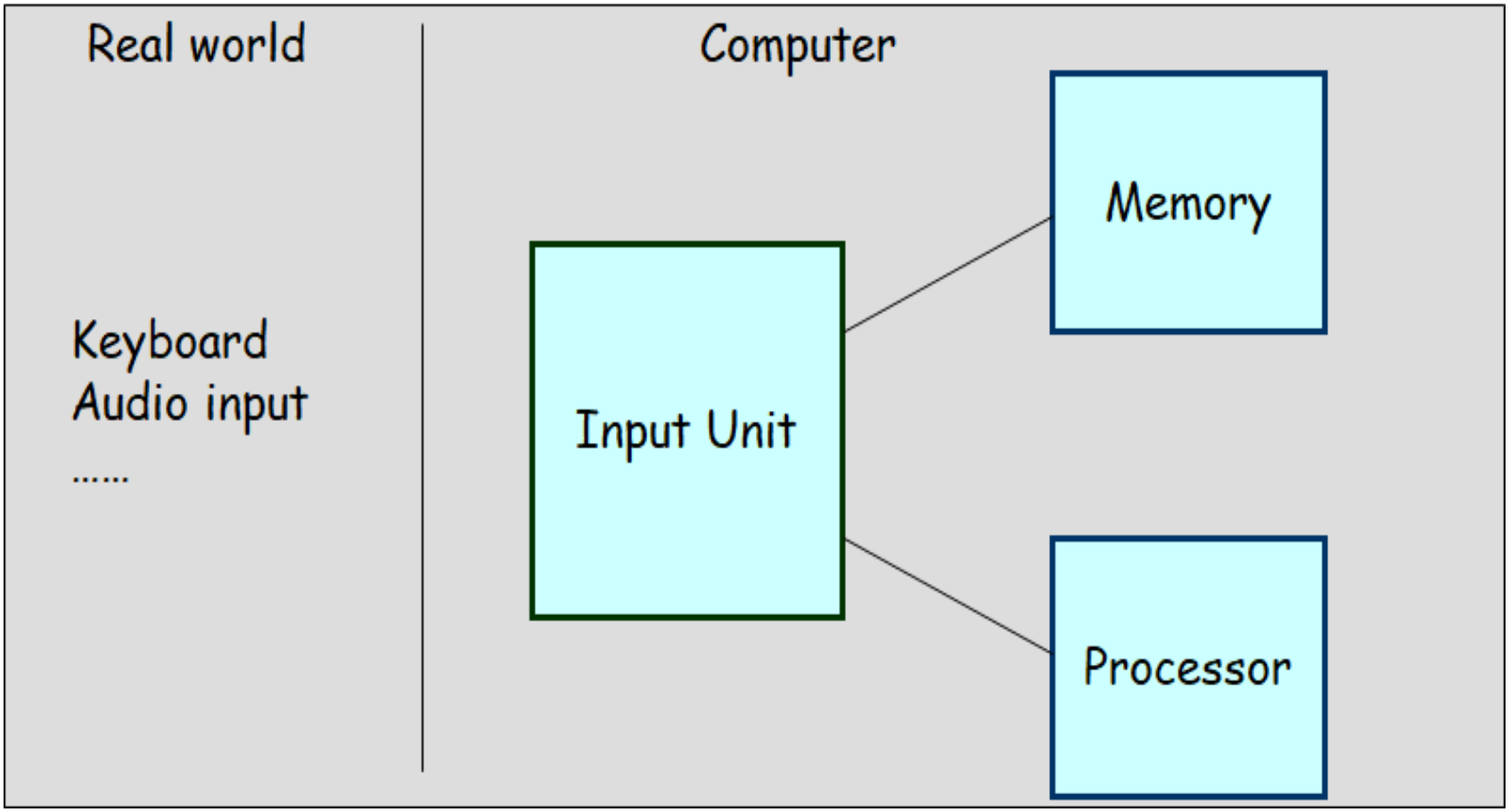
Information in a computer -- Data

- Data are the “operands” upon which instructions operate.
- Data could be:
  - Numbers,
  - Encoded characters.
- Data, in a broad sense means any digital information.
- Computers use data that is encoded as a string of binary digits called bits.

Input unit

- Binary information must be presented to a computer in a specific format. This
- task is performed by the input unit:
  - Interfaces with input devices.
  - Accepts binary information from the input devices.
  - Presents this binary information in a format expected by the computer.
  - Transfers this information to the memory or processor.

Input unit


Real world
Computer
Keyboard
Audio input
.....
Input Unit
Memory
Processor

Memory unit

- Memory unit stores instructions and data.
  - Recall, data is represented as a series of bits.
- Processor reads instructions and reads/writes data from/to the memory during the execution of a program.
  - Group of bits stored or retrieved at a time is termed as “word”
  - Number of bits in a word is termed as the “word length” of a computer.
- In order to read/write to and from memory, a processor should know where to look:
  - “Address” is associated with each word location.

Memory unit

- Processor reads/writes to/from memory based on the memory address:
  - Access any word location in a short and fixed amount of time based on the address.
  - Random Access Memory (RAM) provides fixed access time independent of the location of the word.
  - Access time is known as “Memory Access Time”.
- Memory and processor have to “communicate” with each other in order to read/write information.
  - In order to reduce “communication time”, a small amount of RAM (known as Cache) is tightly coupled with the processor.
  - Modern computers have three to four levels of RAM units with different speeds and sizes:
  - Fastest, smallest known as Cache. Slowest, largest known as Main memory.

Memory unit

- Primary storage of the computer consists of RAM units. Primary storage is insufficient to store large amounts of data and programs.
  - Primary storage can be added, but it is expensive.
- Store large amounts of data on secondary storage devices:
  - Magnetic disks and tapes,
  - Optical disks (CD-ROMS).
  - Access to the data stored in secondary storage in slower, but take advantage of the fact that some information may be accessed infrequently.
- Cost of a memory unit depends on its access time, lesser access time implies higher cost.

Arithmetic and logic unit (ALU)

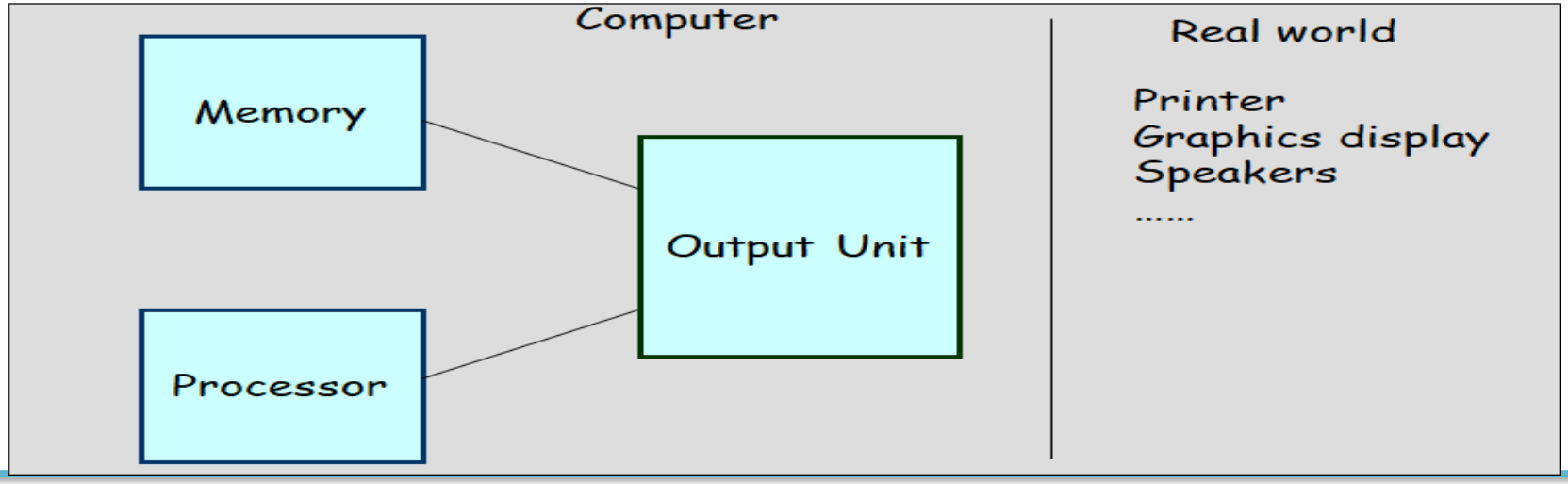
- Operations are executed in the Arithmetic and Logic Unit (ALU).
  - Arithmetic operations such as addition, subtraction.
  - Logic operations such as comparison of numbers.
- In order to execute an instruction, operands need to be brought into the ALU from the memory.
  - Operands are stored in general purpose registers available in the ALU.
  - Access times of general purpose registers are faster than the cache.
- Results of the operations are stored back in the memory or retained in the processor for immediate use.

Output unit

Computers represent information in a specific binary form.

Output units:

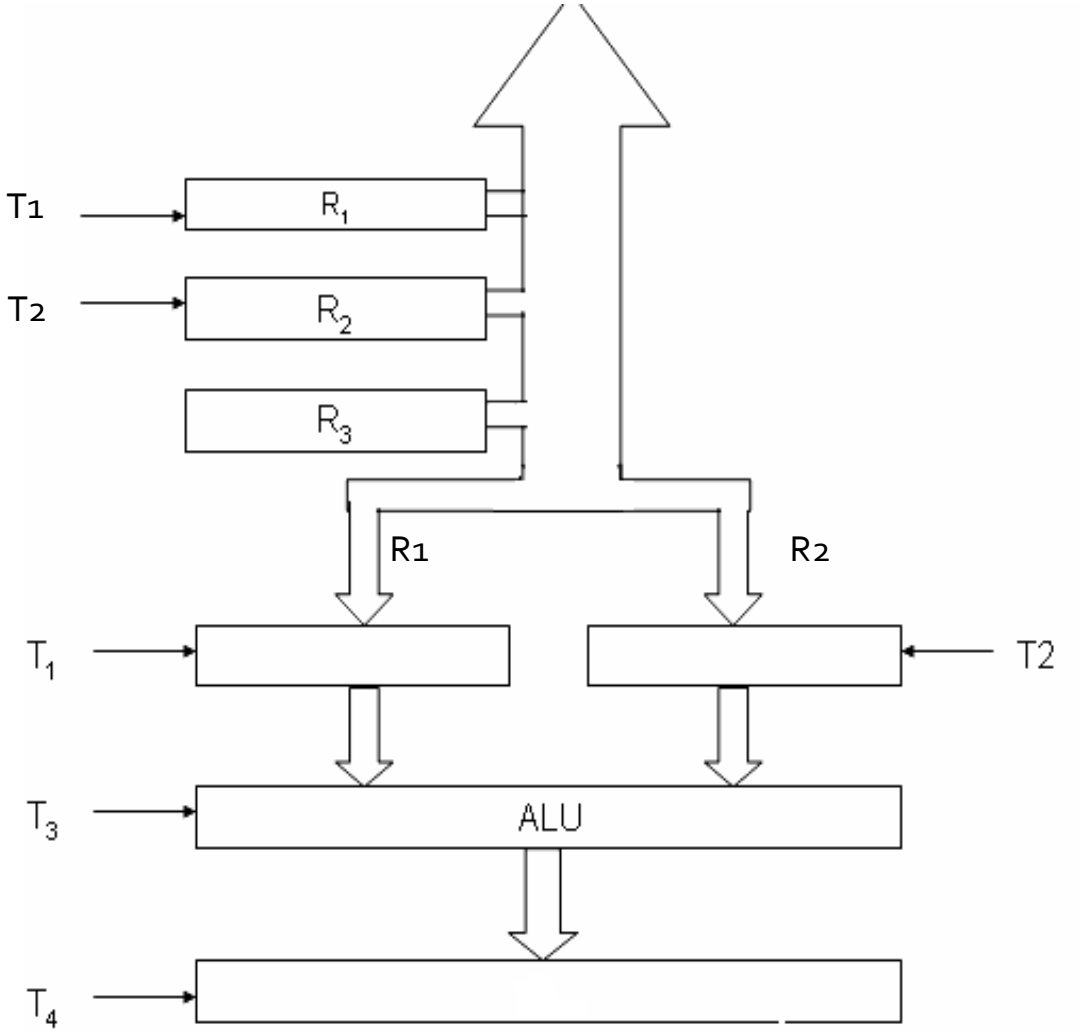
- Interface with output devices.
- Accept processed results provided by the computer in specific binary form.
- Convert the information in binary form to a form understood by an output device.



Control unit

- Operations of Input unit, Memory, ALU and Output unit are coordinated by Control unit.
- Instructions control “what” operations take place (e.g. data transfer, processing).
- Control unit generates timing signals which determines “when” a particular operation takes place.

CU



- Control unit works with a reference signal called Processor clock
- Processor divides the operations into basic steps
- Each basic step is executed in one clock cycle

CU: Example

Example

Add R1, R2

T1 → **Enable R1**

T2 → **Enable R2**

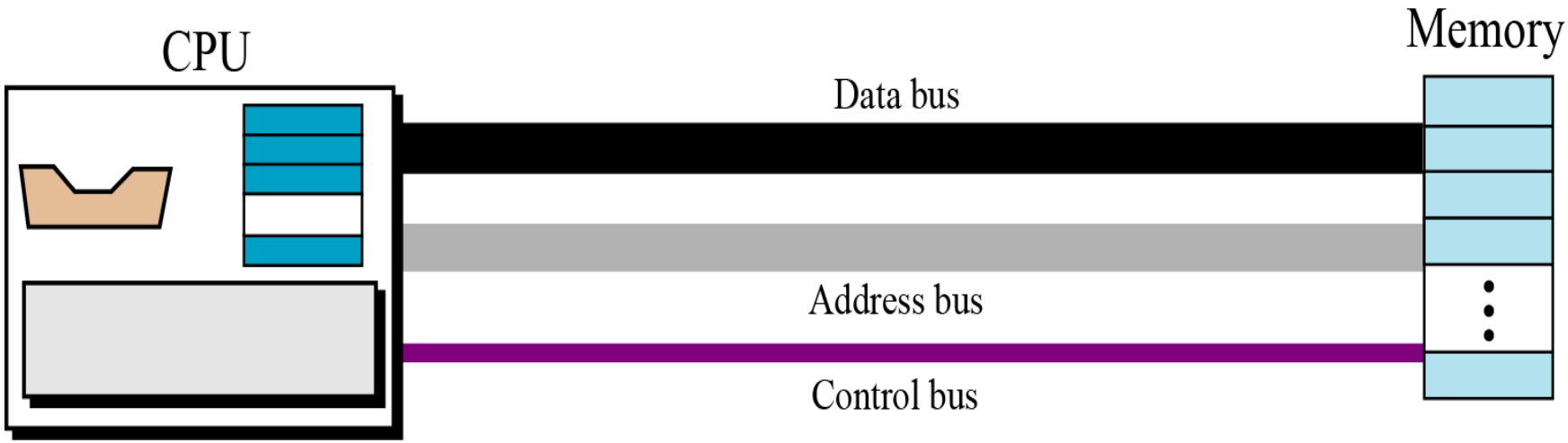
T3 → **Enable ALU for addition operation**

T4 → **Enable out put of ALU to store result of the operation**

Bus Structures

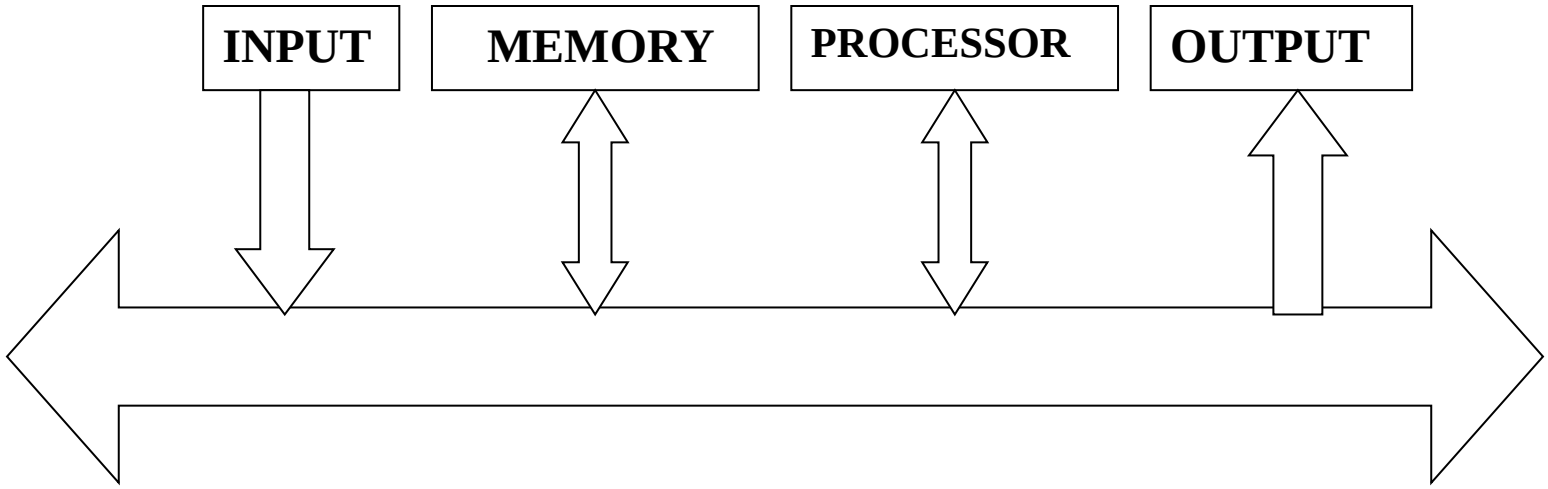
Connecting CPU and memory

- The CPU and memory are normally connected by three groups of connections, each called a **bus**: *data bus*, *address bus* and *control bus*



BUS STRUCTURE

- Group of wires which carries information form CPU to peripherals or vice – versa
- **Single bus structure**: Common bus used to communicate between peripherals and microprocessor



Drawbacks of the Single Bus Structure

- The devices connected to a bus vary widely in their speed of operation.
  - Some devices are relatively slow, such as printer and keyboard.
  - Some devices are considerably fast, such as optical disks.
  - Memory and processor units operate are the fastest parts of a computer.

- Efficient transfer mechanism thus is needed to cope with this problem.
  - A common approach is to include buffer registers with the devices to hold the information during transfers .
  - An another approach is to use two-bus structure and an additional transfer mechanism

TWO BUS STRUCTURE

- One bus can be used to fetch instruction other can be used to fetch data, required for execution.
- The bus is said to perform two distinct functions
- The main advantage of this structure is good operating speed but on account of more cost.

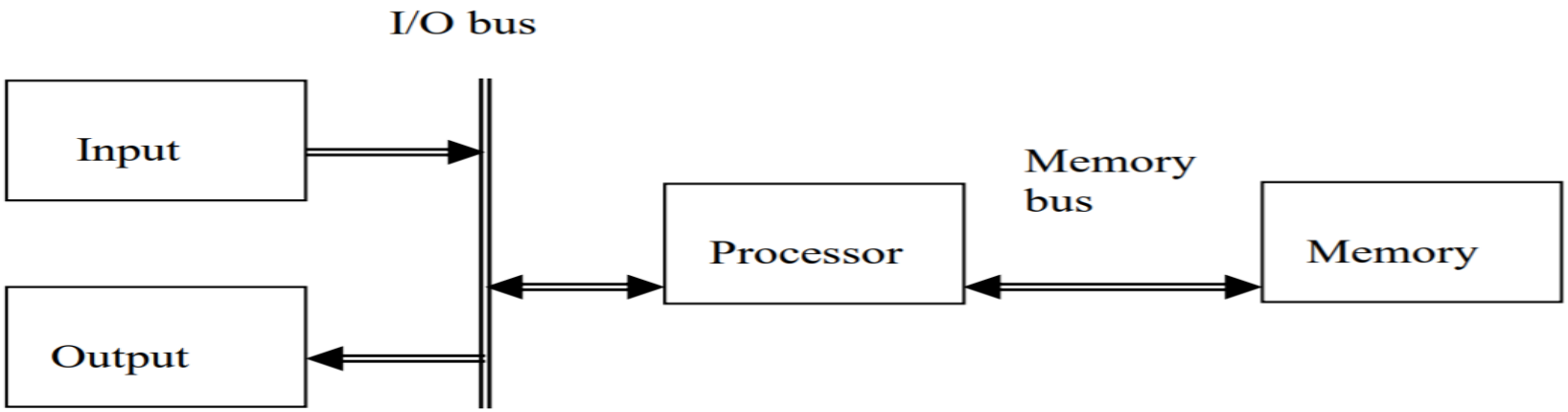
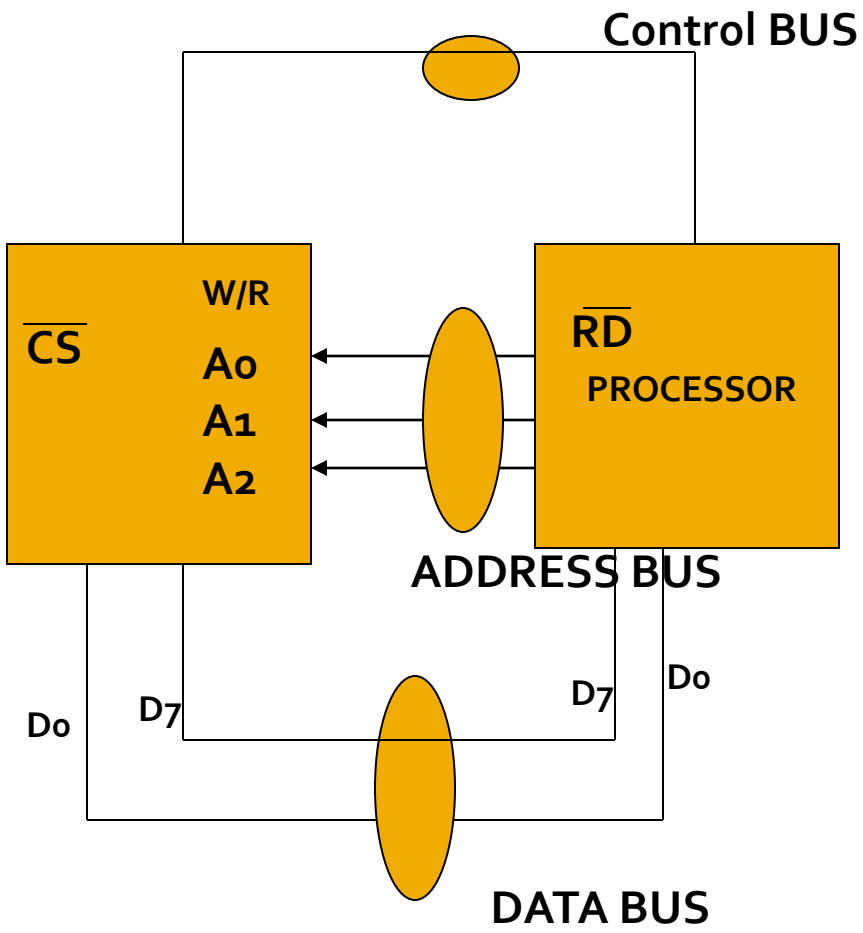


Figure 2.2 Two-bus Structure

MULTI BUS STRUCTURE

- To improve performance multi bus structure can be used.



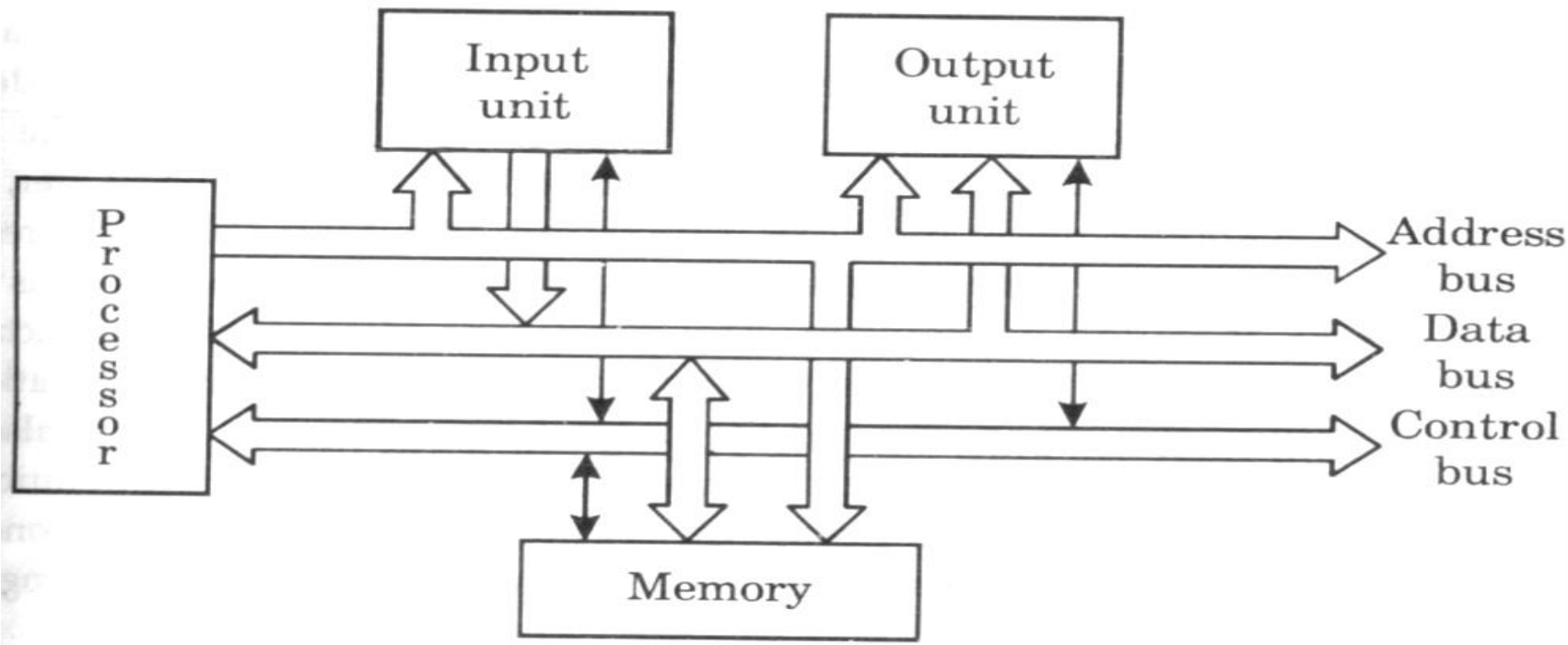
Various Bus Functionalities

- $2^3 = 8$ i.e. 3 address line is required to select 8 location

- In general $2^x = n$ where x number of address lines (address bit) and n is number of location

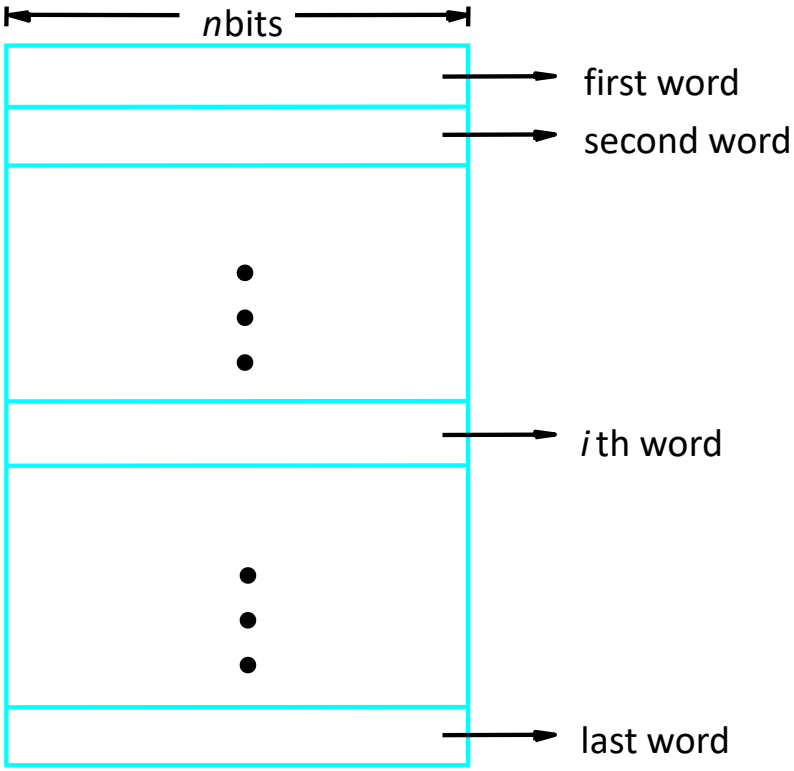
- **Address bus** (Unidirectional): group of wires which carries address information bits from processor to peripherals (16, 20, 24 or more parallel signal lines)
- **Data bus** (Bi-directional): group of wires which carries data information bit from processor to peripherals and vice – versa
- **Control bus (**Bi-directional): group of wires which carries control signals from processor to peripherals and vice – versa

- **Figure below shows address, data and control bus and their connection with peripheral and microprocessor.**



Memory Locations and Addresses

- **Memory consists of many millions of storage cells, each of which can store 1 bit.**
- **Data is usually accessed in *n*-bit groups. *n* is called word length.**
- **The memory of a computer can be schematically represented as a collection of words as shown in Figure 1.**



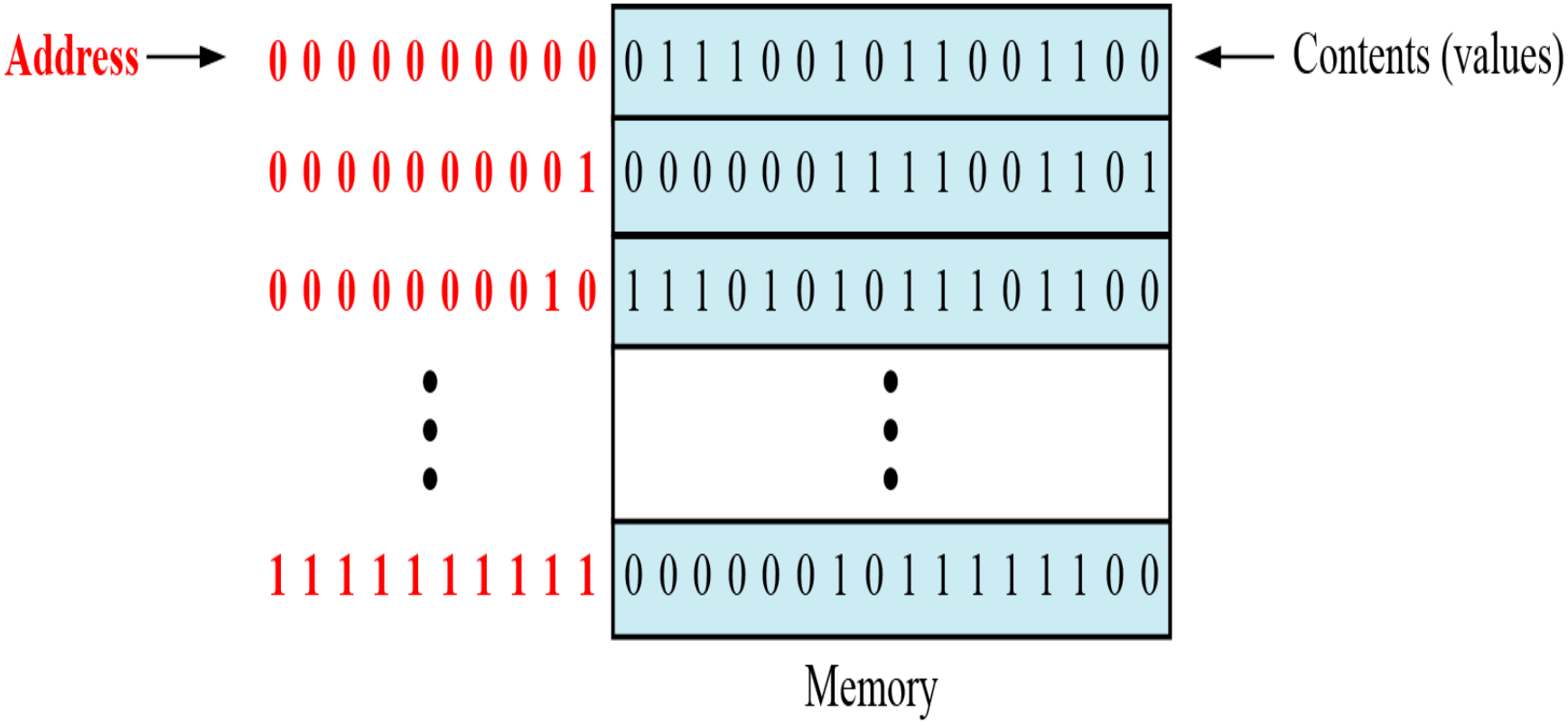
Main Memory words.

Memory Locations And Addresses

- **Main memory** is the second major subsystem in a computer. It consists of a collection of storage locations, each with a unique identifier, called an **address**.

- Data is transferred to and from memory in groups of bits called **words**. A word can be a group of 8 bits, 16 bits, 32 bits or 64 bits (and growing).

- If the word is 8 bits, it is referred to as a **byte**. The term “byte” is so common in computer science that sometimes a 16-bit word is referred to as a 2-byte word, or a 32-bit word is referred to as a 4-byte word.


Address
0 0 0 0 0 0 0 0 0 0
0 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0
Contents (values)
0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 1 1 1 0 0 1 1 0 1
0 0 0 0 0 0 0 0 1 0
1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 0
1 1 1 1 1 1 1 1 1 1
0 0 0 0 0 0 1 0 1 1 1 1 1 1 0 0
Memory

Main memory

Address space

- To access a word in memory requires an identifier. Although programmers use a name to identify a word (or a collection of words), at the hardware level each word is identified by an address.

- The total number of uniquely identifiable locations in memory is called the address space.

- For example, a memory with 64 kilobytes (16 address line required) and a word size of 1 byte has an address space that ranges from 0 to 65,535.

Table 5.1 Memory units

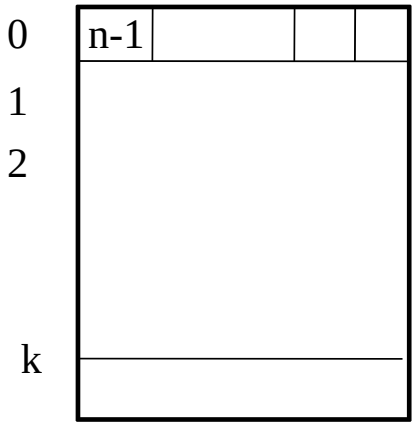
Unit	*Exact Number of Bytes*	*Approximation*
kilobyte	2^{10} (1024) bytes	10^3 bytes
megabyte	2^{20} (1,048,576) bytes	10^6 bytes
gigabyte	2^{30} (1,073,741,824) bytes	10^9 bytes
terabyte	2^{40} bytes	10^{12} bytes

Memory addresses are defined using unsigned binary integers.

Memory

- Holds both instructions and data
- With *k* address bits and *n* bits per location address
- n is typically 8 (byte), 16 (word), 32 (long word), ….

k	Number of locations
10	2^{10} = 1024 = 1K
16	2^{16} = 65,536 = 64K
20	2^{20} = 1,048,576 = 1M
24	2^{24} = 16,777,216 = 16M



Example 1

A computer has 32 MB (megabytes) of memory. How many bits are needed to address any single byte in memory?

Solution

The memory address space is 32 MB, or 2^{25} ($2^5 \times 2^{20}$). This means that we need $\log_2 2^{25}$, or **25 bits**, to address each byte.

Example 2

A computer has 128 MB of memory. Each word in this computer is eight bytes. How many bits are needed to address any single word in memory?

Solution

The memory address space is 128 MB, which means 2^{27}. However, each word is eight (2^3) bytes, which means that we have 2^{24} words. This means that we need $\log_2 2^{24}$, or **24 bits**, to address each word.

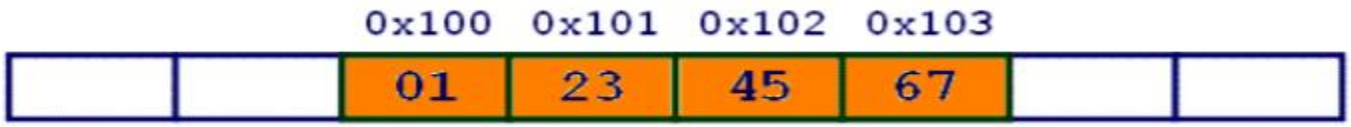
Memory Operations

- Two basic operations requires in memory access
  - Load operation (Read or Fetch)-Contents of specified memory location are read by processor
  - Store operation (Write)- Data from the processor is stored in specified memory location

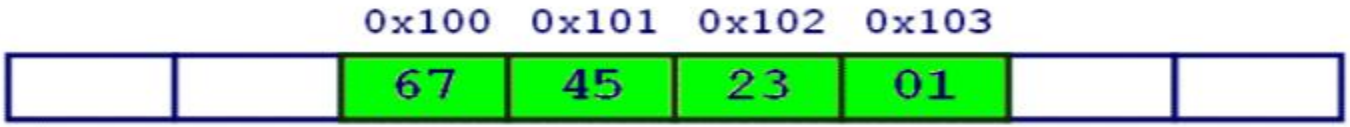
Assignment of Byte Address

- **Big-endian** is an order in which the "bigend" (most significant value in the sequence) is stored first (at the lowest storage address).
- **Little-endian** is an order in which the “Little end" (least significant value in the sequence) is stored first (at the lowest storage address).

Little vs. Big Endian Notations



Big Endian

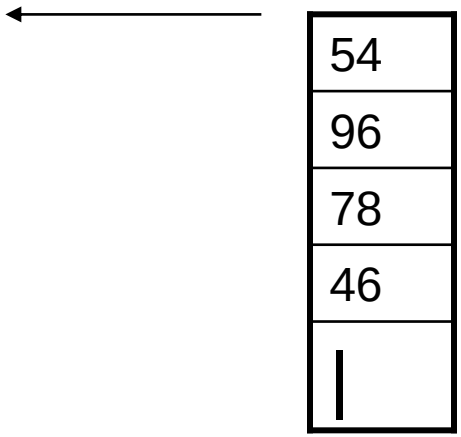


Little Endian

Eg: 46,78,96,54 (32 bit data)

H BYTE L BYTE

8000
8001
8002
8003
8004



Basic Operational Concepts

Basic Function of Computer

- To Execute a given task as per the appropriate program
- Program consists of list of instructions stored in memory

A Typical Instruction

Add R0, LOCA

- Add the operand at memory location LOCA to the operand in a register R0 in the processor.
- Place the sum into register R0.
- The original contents of LOCA are preserved.
- The original contents of R0 is overwritten.
- Instruction is fetched from the memory into the processor – the operand at LOCA is fetched and added to the contents of R0 – the resulting sum is stored in register R0.

Load R2, LOC

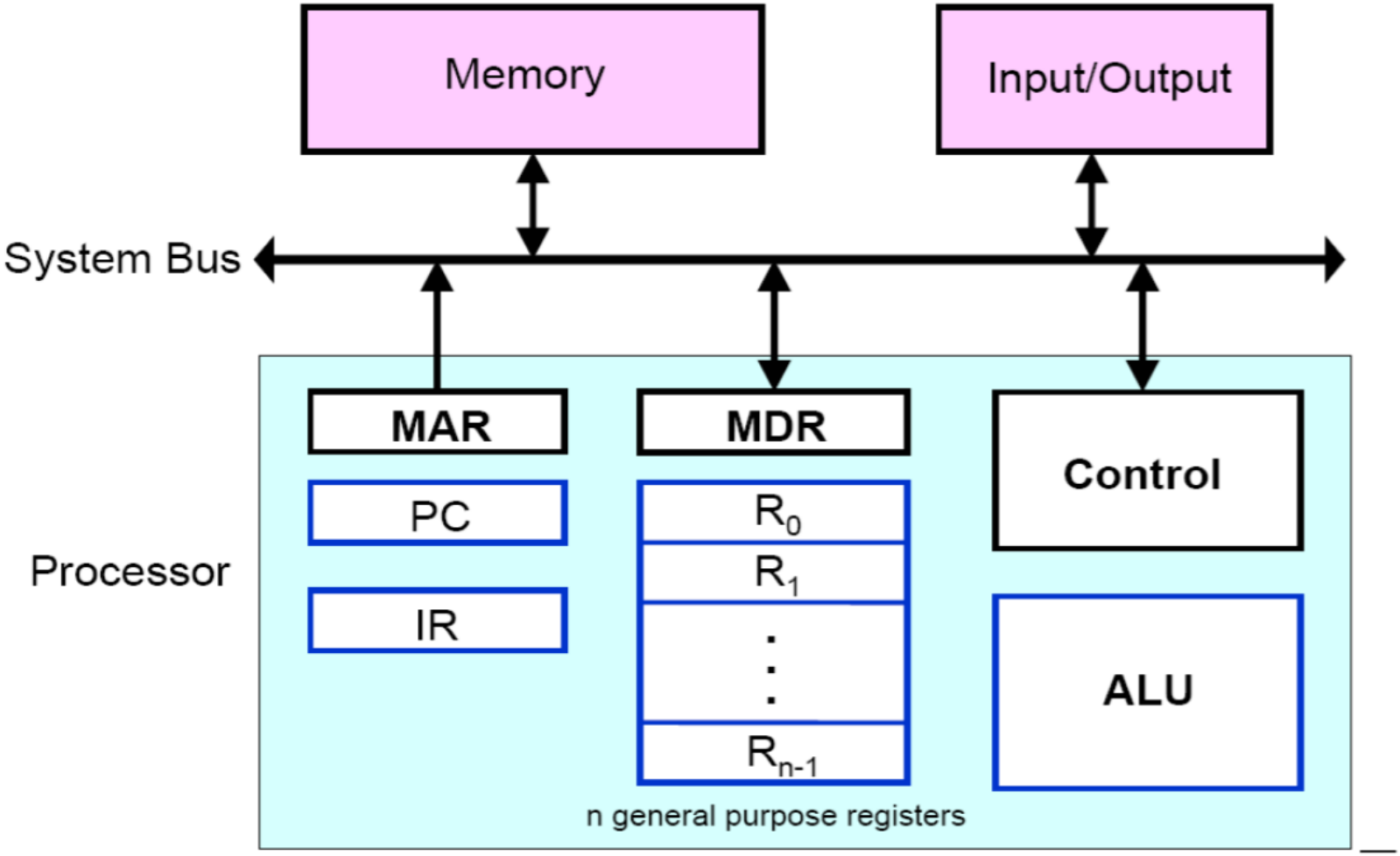
- This instruction reads the contents of a memory location by the label LOC and loads them into processor register R2.
- The original contents of location LOC are preserved, whereas those of register R2 are overwritten.
- Execution of this instruction requires several steps.
- First, the instruction is fetched from the memory into the processor.
- Next, the operation to be performed is determined by the control unit.
- The operand at LOC is then fetched from the memory into the processor.
- Finally, the operand is stored in register R2

Add R4, R2, R3

- adds the contents of registers R2 and R3, then places their sum into register R4.
- The operands in R2 and R3 are not altered, but the previous value in R4 is overwritten by the sum.

Store R4, LOC

- This instruction copies the operand in register R4 to memory location LOC.
- The original contents of location LOC are overwritten, but those of R4 are preserved.
- For Load and Store instructions, transfers between the memory and the processor are initiated by sending the address of the desired memory location to the memory unit and asserting the appropriate control signals.
- The data are then transferred to or from the memory.



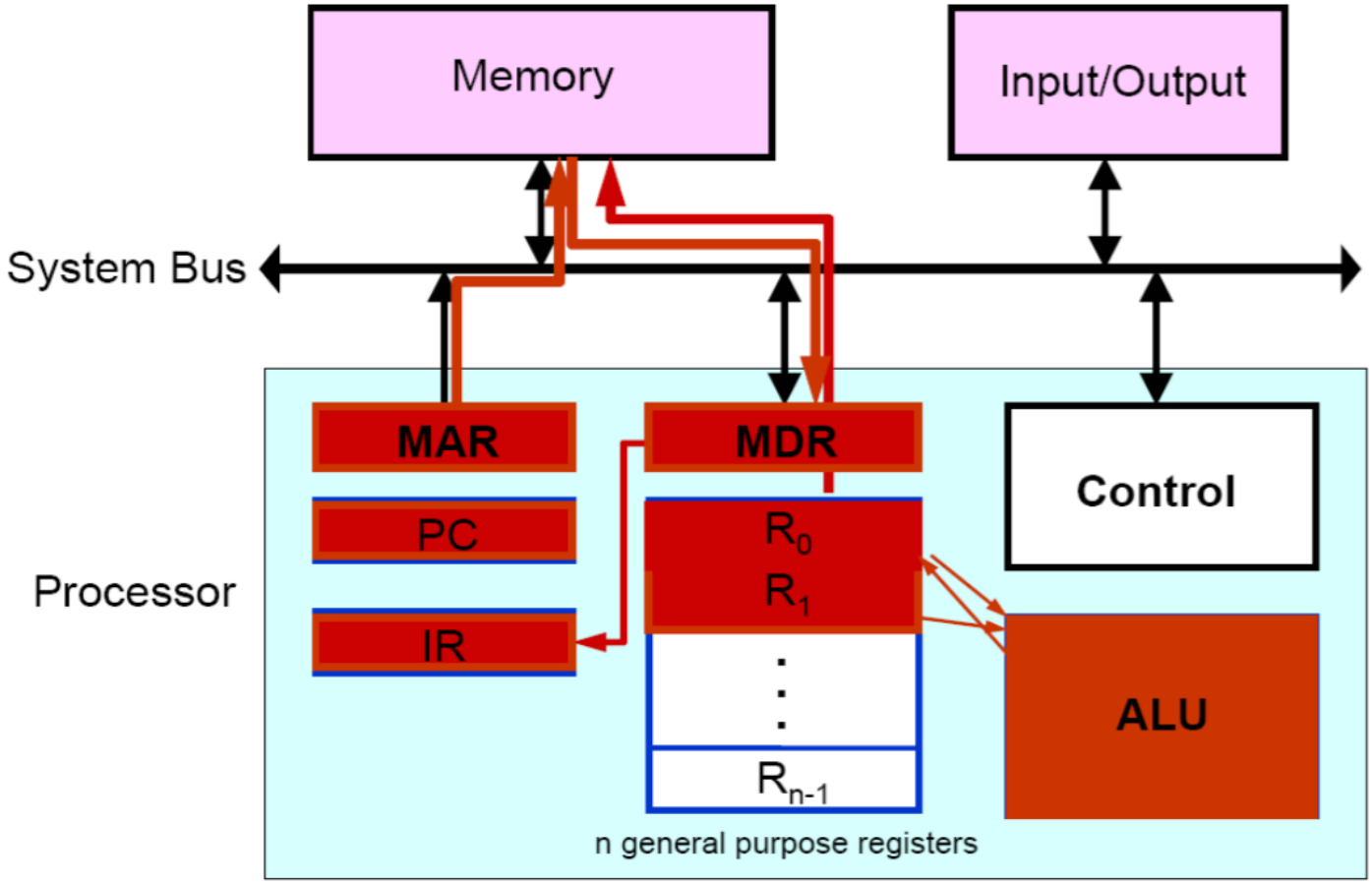
Connections between Processor and memory

Registers: Registers are fast stand-alone storage locations that hold data temporarily. Multiple registers are needed to facilitate the operation of the CPU.

Some of these registers are:

- MAR (Memory Address Register) and MDR (Memory Data Register) : To handle the data transfer between main memory and processor. MAR-Holds addresses, MDR-Holds data
- Instruction register (IR) : Hold the Instructions that is currently being executed
- Program counter (PC) : Points to the next instructions that is to be fetched from memory
- General-purpose Registers: are used for holding data, intermediate results of operations. They are also known as scratch-pad registers.

Basic Operational Concepts



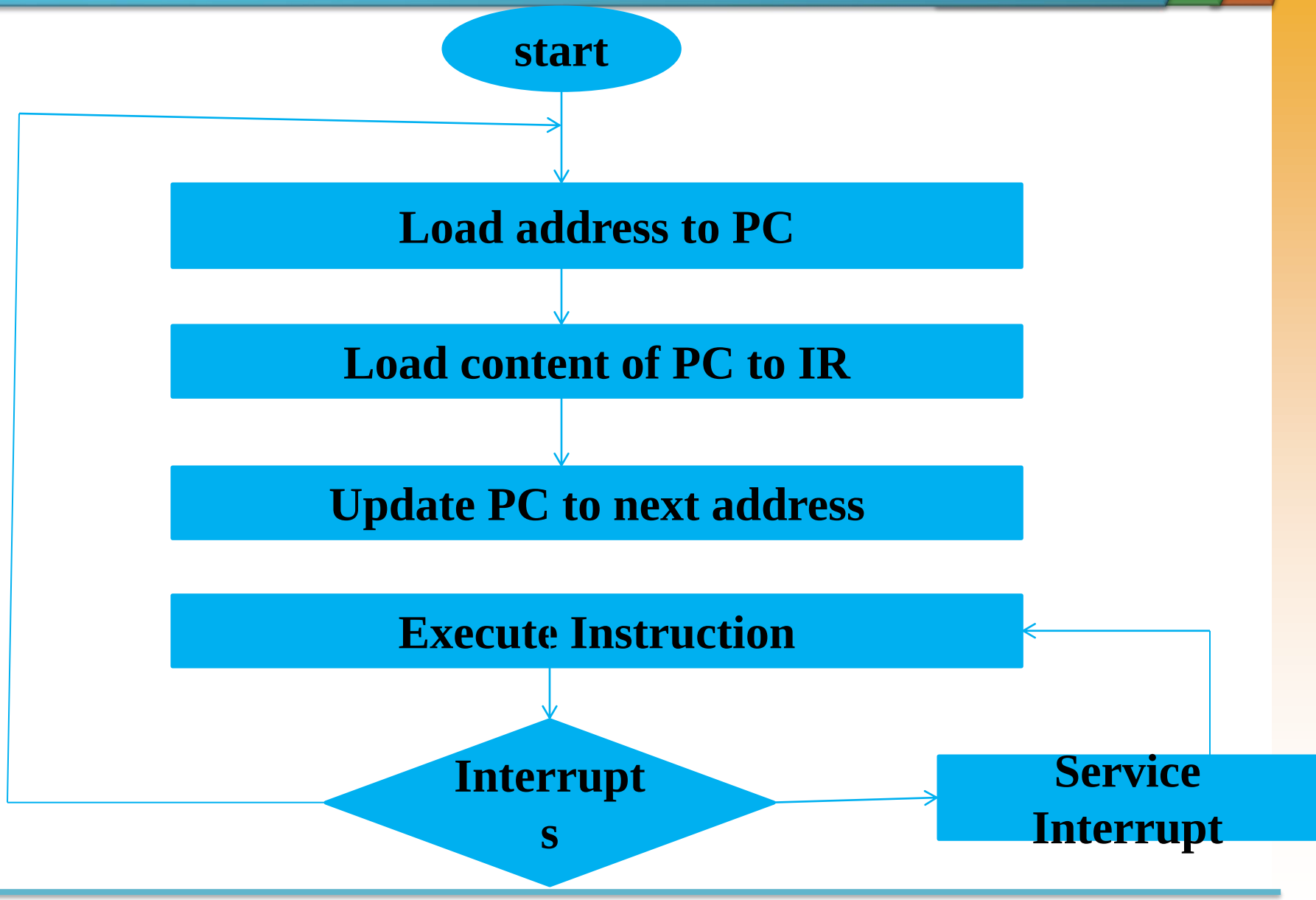
Instruction Fetch

- Program gets into the memory through an input device.
- Execution of a program starts by setting the PC to point to the first instruction of the program.
- The contents of PC are transferred to the MAR and a Read control signal is sent to the memory.
- The addressed word (here it is the first instruction of the program) is read out of memory and loaded into the MDR.
- The contents of MDR are transferred to the IR for instruction decoding

Instruction Fetch

- The operation field of the instruction in IR is examined to determine the type of operation to be performed by the ALU.
- The specified operation is performed by obtaining the operand(s) from the memory locations or from GP registers.

1) Fetching the operands from the memory requires sending the memory location address to the MAR and initiating a Read cycle.
2) The operand is read from the memory into the MDR and then from MDR to the ALU.
3) The ALU performs the desired operation on one or more operands fetched in this manner and sends the result either to memory location or to a GP register.
4) The result is sent to MDR and the address of the location where the result is to be stored is sent to MAR and Write cycle is initiated.


start
Load address to PC
Load content of PC to IR
Update PC to next address
Execute Instruction
Interrupt
s
Service
Interrupt

Interrupt

- An interrupt is a request from I/O device for service by processor
- Processor provides requested service by executing interrupt service routine (ISR)
- Contents of PC, general registers, and some control information are stored in memory .
- When ISR completed, processor restored, so that interrupted program may continue

Instruction and Instruction Sequencing

Introduction

Instruction: is command to the microprocessor to perform a given task on specified data.

Instruction Set: The entire group of these instructions are called instruction set.

Instruction sequencing : The order in which the instructions in a program are carried out.

A computer must have instructions capable of performing 4 types of operation

- Data transfer between memory and processor register
- Arithmetic and logic operation
- Program sequencing and control
- I/O transfer

Register transfer notation (RTN)

- Transfer between processor registers & memory, between processor register & I/O devices
- Memory locations, registers and I/O register names are identified by a symbolic name in uppercase alphabets
- LOC,PLACE,MEM are the address of memory location
- R1 , R2,… are processor registers
- DATA_IN, DATA_OUT are I/O registers
  - Contents of location is indicated by using square brackets [ ]
  - RHS of RTN always denotes a values, and is called Source
  - LHS of RTN always denotes a symbolic name where value is to be stored and is called destination
  - Source contents are not modified
  - Destination contents are overwritten

Examples of RTN statements

- R2 ← [LOCN]
- R4 ← [R3] +[R2]

Data transfer Instructions

They are also called copy instructions.

Some instructions in 8086:

- MOV -Copy from the source to the destination
- LDA - Load the accumulator
- STA - Store the accumulator
- PUSH - Push the register pair onto the stack
- POP - Pop off stack to the register pair

Data Manipulation Instructions

To perform the operations by the ALU

Three categories:

- Arithmetic Instructions
- Logical and bit manipulation instructions
- Shift instructions

Arithmetic Instructions

Used to perform arithmetic operations

Some instructions in 8086:

- **INC** ➔ Increment the data by 1
- **DEC** ➔ Decreases data by 1
- **ADD** ➔ perform sum of data
- **ADC** ➔ Add with carry bit.
- **MUL** ➔ perform multiplication

Logical and Bit Manipulation Instructions

Used to perform logical operations

Some instructions are:

- AND ➔ bitwise AND operation
- OR ➔ bitwise AND operation
- NOT ➔ invert each bit of a byte or word
- XOR ➔ Exclusive-OR operation over each bit

Shift Instructions

- Used for shifting or rotating the contents of the register

Some instructions are:

- SHR ➔ shift bits towards the right and put zero(S) in MSBs
- ROL ➔ rotate bits towards the left, i.e. MSB to LSB and to Carry Flag [CF]
- RCL ➔ rotate bits towards the left, i.e. MSB to CF and CF to LSB.

Instruction Formats

Types of instruction based on the address field

- An instruction in computer comprises of groups called fields.
- These field contains different information
- The most common fields are:
  - Operation field : specifies the operation to be performed like addition.
  - Address field : contain the location of operand
  - Mode field : specifies how to find the operand

An instruction is of various length depending upon the number of addresses it contain.

On the basis of number of address, instruction are classified as:

- **Zero Address Instructions**
- **One Address Instructions**
- **Two Address Instructions**
- **Three Address Instructions**

Zero Address Instructions

Used in stack-based computers which do not use address field in instruction

- Location of all operands are defined implicitly
- Operands are stored in a structure called pushdown stack

Example: Evaluate (A+B) * (C+D)

Using Zero-Address instruction

1.	PUSH	A	; TOS ← A
2.	PUSH	B	; TOS ← B
3.	ADD		; TOS ← (A + B)
4.	PUSH	C	; TOS ← C
5.	PUSH	D	; TOS ← D
6.	ADD		; TOS ← (C + D)
7.	MUL		; TOS ← (C+D)*(A+B)
8.	POP	X	; M[X] ← TOS

One address Instruction

Syntax- Operation source/destination

- In this type either a source or destination operand is mentioned in the instruction
- Other operand is implied to be a processor register called Accumulator
- Eg: ADD B (general)

  - Load D; ACC [memlocation _D]
  - ADD E; ACC (ACC) +(E)
  - STORE F; memlocation_ F (ACC)

One address Instruction

- This uses an implied ACCUMULATOR register for data manipulation.
- Operation Destination: One operand is in accumulator and other is in register or memory location.

Example: Evaluate (A+B) ∗ (C+D)

Using One-Address Instruction

1.	LOAD	A	; AC ← M[A]
2.	ADD	B	; AC ← AC + M[B]
3.	STORE	T	; M[T] ← AC
4.	LOAD	C	; AC ← M[C]
5.	ADD	D	; AC ← AC + M[D]
6.	MUL	T	; AC ← AC ∗ M[T]
7.	STORE	X	; M[X] ← AC

Two Address Instruction

Syntax: Operation source, destination

Eg: 1. MOVE E, F; 2. MOVE D,F; 3. ADD D, F;

Perform ADD A,B,C using 2 instructions

- MOVE B,C
- ADD A,C

Disadvantage: Single instruction is not sufficient to perform the entire operation.

Two Address Instruction

- This is common in commercial computers.

Eg: Operation Source, Destination

- Here two address can be specified in the instruction.
- Example: Evaluate (A+B) ∗ (C+D)

Using Two address Instruction:

1. MOV R1,A ; R1=M[A]
2. ADD R1,B ;R1=R1+M[B]
3. MOV R2,C ;R2=M[C]
4. ADD R2,D ;R2=R2+M[D]
5. MUL R1,R2 ; R1=R1*R2
6. MOV X,R1 ; M[X]=R1

Three Address Instruction

Syntax: Operation source 1, source 2, destination

- Eg: ADD D,E,F where D,E,F are memory location

- Advantage: Single instruction can perform the complete operation
- Disadvantage: Instruction code will be too large to fit in one word location in memory

Three Address Instruction

- This has three address field to specify a register or a memory location.
- Program created are much short in size creation of program much easier

It does not mean that program will run much faster. The program execution speed indeed depends on:

- How fast this long instruction can be fetched from the memory,
- How fast such lengthy instructions can be decoded and executed by the processor (here, the term “Fast” corresponds to the either Machine cycles or T-cycles).

Just recall the difference in the length of a Python program and C-program and recall which one is fast w.r.t. execution in the real world.

Example:

Example: Evaluate (A+B) ∗ (C+D)

Using Three address Instruction

1. ADD R1,A,B; R1=M[A]+M[B]
2. ADD R2,C,D; R2=M[C]+M[D]
3. MUL X,R1,R2; M[X]=R1*R2

Instruction Cycle

- The basic operational process of a computer.
- Also known as **fetch-decode-execute cycle**
- This process is repeated continuously by CPU from boot up to shut down of computer.

Steps that occur during an instruction cycle:

1. **Fetch the Instruction**
2. **Decode the Instruction**
3. **Read the Effective Address and fetch the operands**
4. **Execute the Instruction**

1. **Fetch the Instruction:** The instruction is fetched from memory address that is stored in PC(Program Counter) and stored in the (instruction register) IR.

At the end of the fetch operation, PC is incremented by 1 and it then points to the next instruction to be executed.

2. **Decode the Instruction:** The instruction in the IR is decoded(Interpreted).

3. Read the Effective Address: If the instruction has an indirect address, the effective address is read from the memory.

Otherwise, operands are directly read in case of immediate operand instruction.

4. Execute the Instruction: The Control Unit passes the information in the form of control signals to the functional unit of CPU. The result generated is stored in main memory or sent to an output device.

The cycle is then repeated by fetching the next instruction. This way, the instruction cycle is repeated continuously.

Instruction Execution & Straightline Sequencing

Instruction execution needs the following steps:

- PC (program counter) register of the processor gives the address of the instruction which needs to be fetched from the memory.
- If the instruction is fetched then, the instruction opcode is decoded. On decoding, the processor identifies the number of operands. If there is any operand to be fetched from the memory, then that operand address is calculated.
- Operands are fetched from the memory. If there is more than one operand, then the operand fetching process may be repeated (i.e. address calculation and fetching operands).
- After this, the data operation is performed on the operands, and a result is generated.

- If the result has to be stored in a register, the instructions end here.
- If the destination is memory, then first the destination address has to be calculated. Then the result is then stored in the memory. If there are multiple results which need to be stored inside the memory, then this process may repeat (i.e. destination address calculation and store result).
- Now the current instructions have been executed. Side by side, the PC is incremented to calculate the address of the next instruction.
- The above instruction cycle then repeats for further instructions.

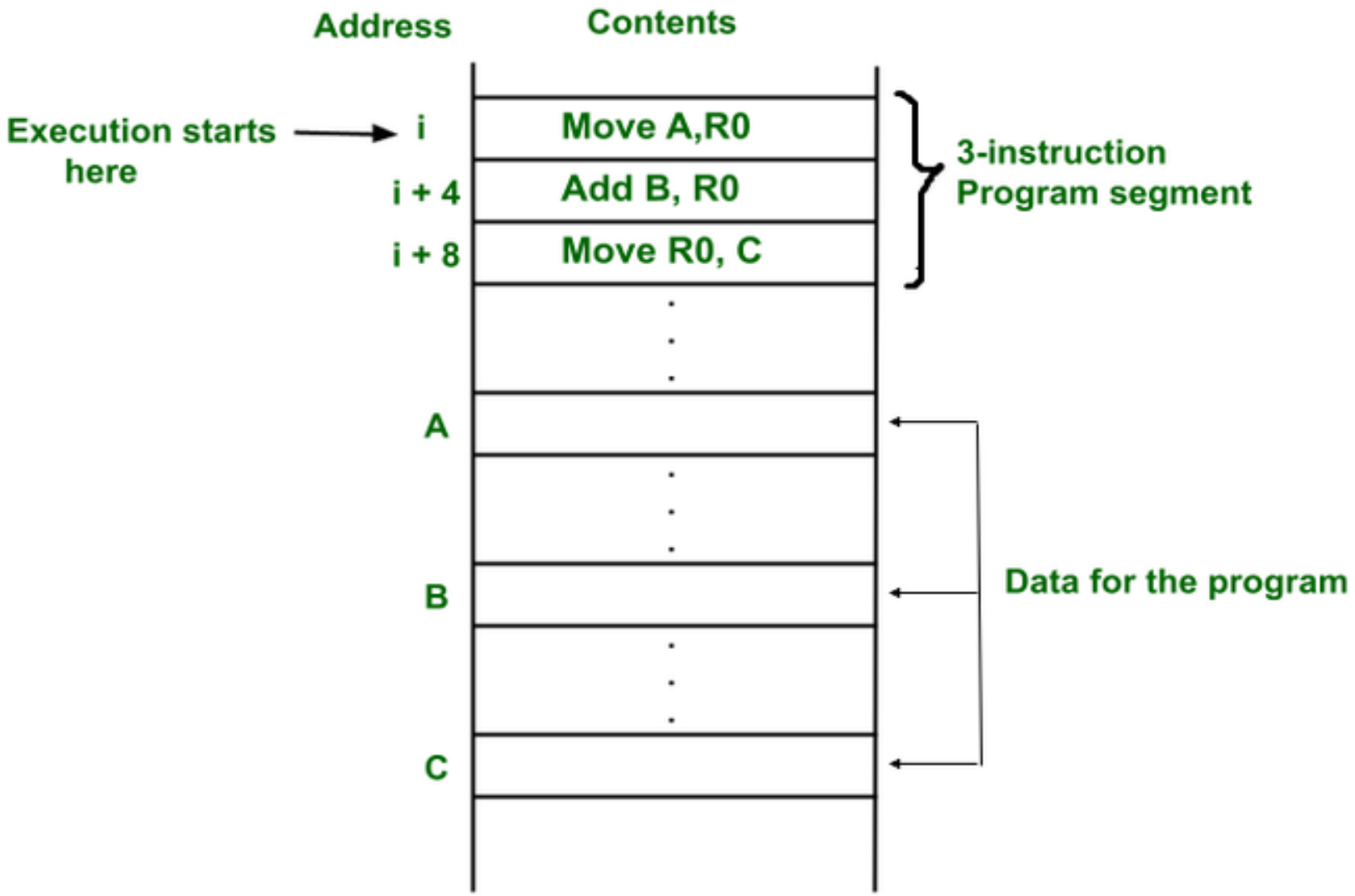
Straight Line Sequencing

- Straight line sequencing means the instruction of a program is executed in a sequential manner(i.e. every time PC is incremented by a fixed offset).
- And no branch address is loaded on the PC.

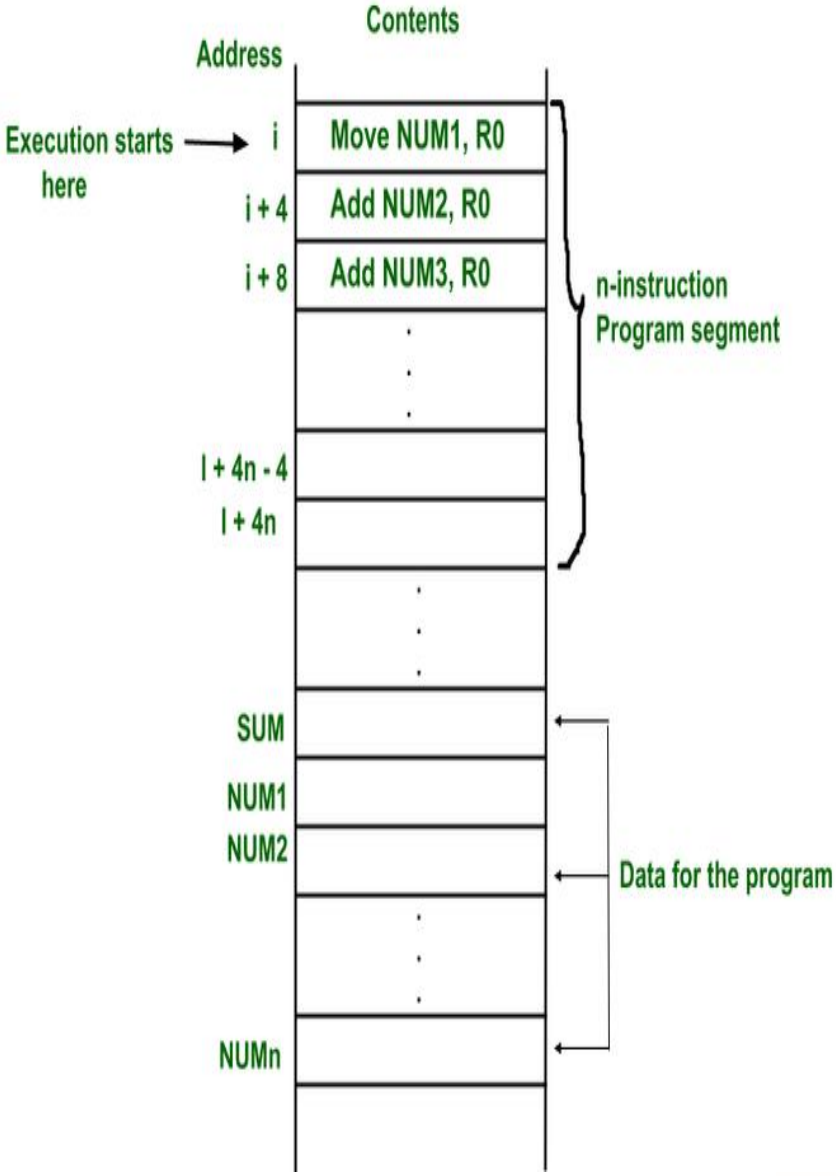
Example 1:

- Programs and Data are stored in the same memory, i.e. Von-Neumann architecture. (Note, here data refers to the variables in a program, where prog. is the actual instructions within it)
- First instruction of a program is stored at an address '*i*'.
- PC gives address '*i*' and instruction stored at that address '*i*' is fetched from the memory and then decoded.
- Then operand '*A*' is fetched from the memory and stored in a temporary register.
- The instruction is executed (i.e. content of address A is copied into processor register R0).

- Here, the ALU data path is supplemented by few registers (not a reg. profile register) which holds the data quite temporarily.
- The PC gets incremented by 4 (i.e. it contains the address of the next instruction) because the instruction and memory segment is of 4 bytes.
- So, the instruction at address '*i*' is executed.
- Every time, the PC is incremented by 4 and the program is executing in a sequential manner. And this process is called straight line sequencing.


Address
Contents
Execution starts here
i
Move A,R0
i + 4
Add B, R0
i + 8
Move R0, C
3-instruction Program segment
A
B
C
Data for the program

- The addresses of the memory locations containing the n numbers are represented as NUM1, NUM2, ….. NUMn (i.e. NUM1 address includes first number).
- The first number is stored into processor register R0.
- And every other number is added to register R0. Finally, when the program ends(i.e. n numbers are added, the result is placed in memory location SUM

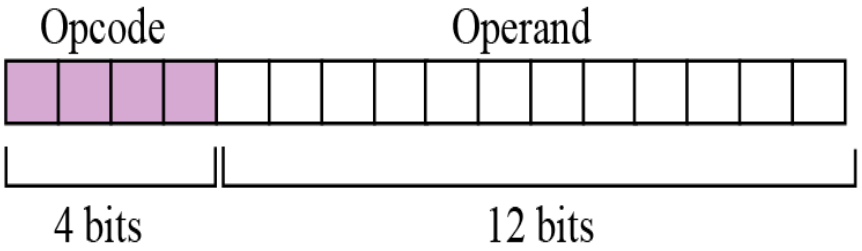


Condition Codes (Flags)

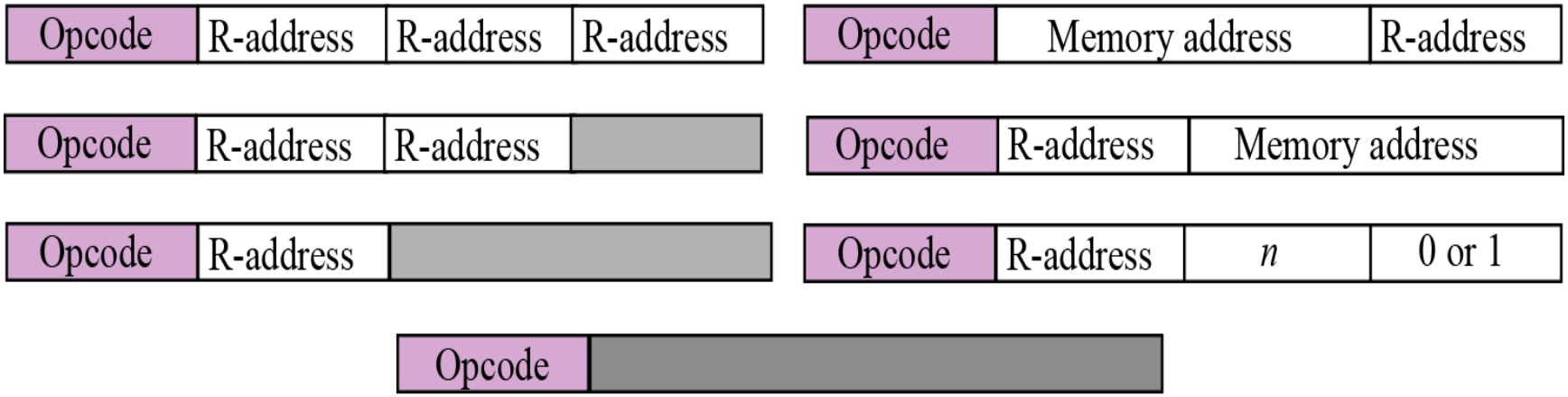
- The processor keeps track of information about the results of various operations for use by subsequent conditional branch instructions

N – Negative	1 if results are Negative
	0 if results are Positive
Z – Zero	1 if results are Zero
	0 if results are Non zero
V – Overflow	1 if arithmetic overflow occurs
	0 non overflow occurs
C – Carry	1 if carry and from MSB bit
	0 if there is no carry from MSB bit

Format and Different Instruction Types



a. Instruction format



b. Instruction types

Addressing Modes

Different ways in which the location of the operand is specified in an instruction is referred as addressing modes

The purpose of using addressing mode is:

To give the programming versatility to the user.

To reduce the number of bits in addressing field of instruction.

Types of Addressing Modes

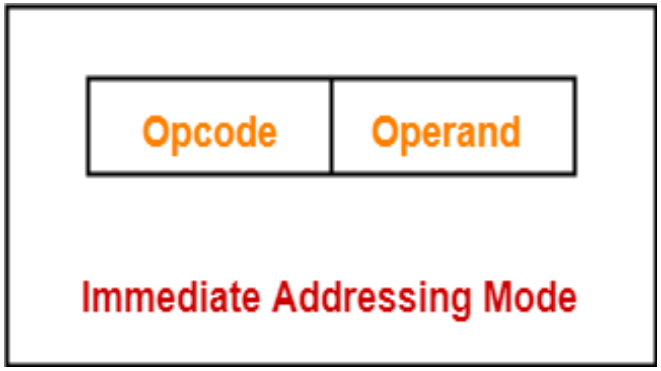
- Immediate Addressing
- Direct Addressing
- Indirect Addressing
- Register Addressing
- Register Indirect Addressing
- Relative Addressing
- Indexed Addressing
- Auto Increment
- Auto Decrement

Immediate Addressing

- Operand is given explicitly in the instruction

E.g. ADD 5

  - Add 5 to contents of accumulator
  - 5 is operand



- No memory reference to fetch data
- Fast
- Limited range
  - MOV AL, 25H; Immediate addressing: AL=25
  - MOV AX, 2345H; AX=2345; AX=> AH=23 and AL=45

Direct Addressing

- Address field contains address of operand
- Effective address (EA) = address field (A)

E.g. ADD A

  - Add contents of cell A to accumulator
  - Look in memory at address A for operand

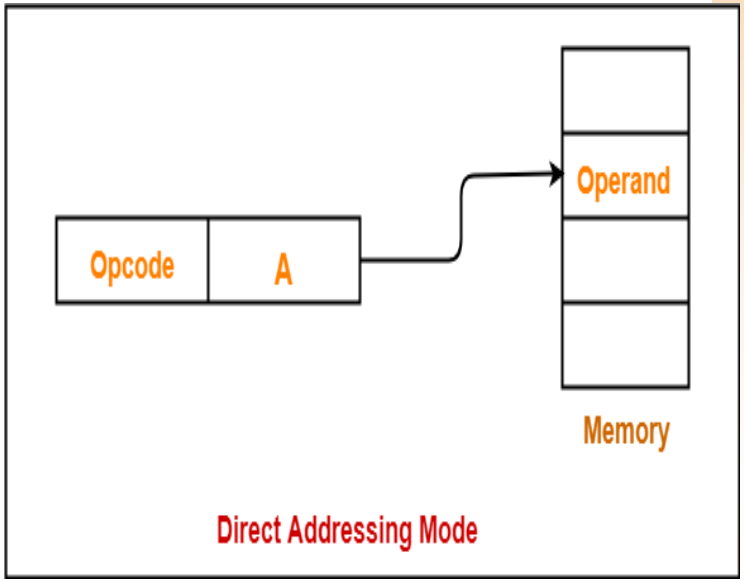
- Single memory reference to access data
- No additional calculations to work out effective address
- Limited address space

MOV AL, DATA1; Direct Addressing AL = 23

MOV AX, DATA2; AX = 1234

MOV DATA3, AL; DATA3 = 23

MOV DATA4,AX ; DATA4 = 1234



Indirect Addressing

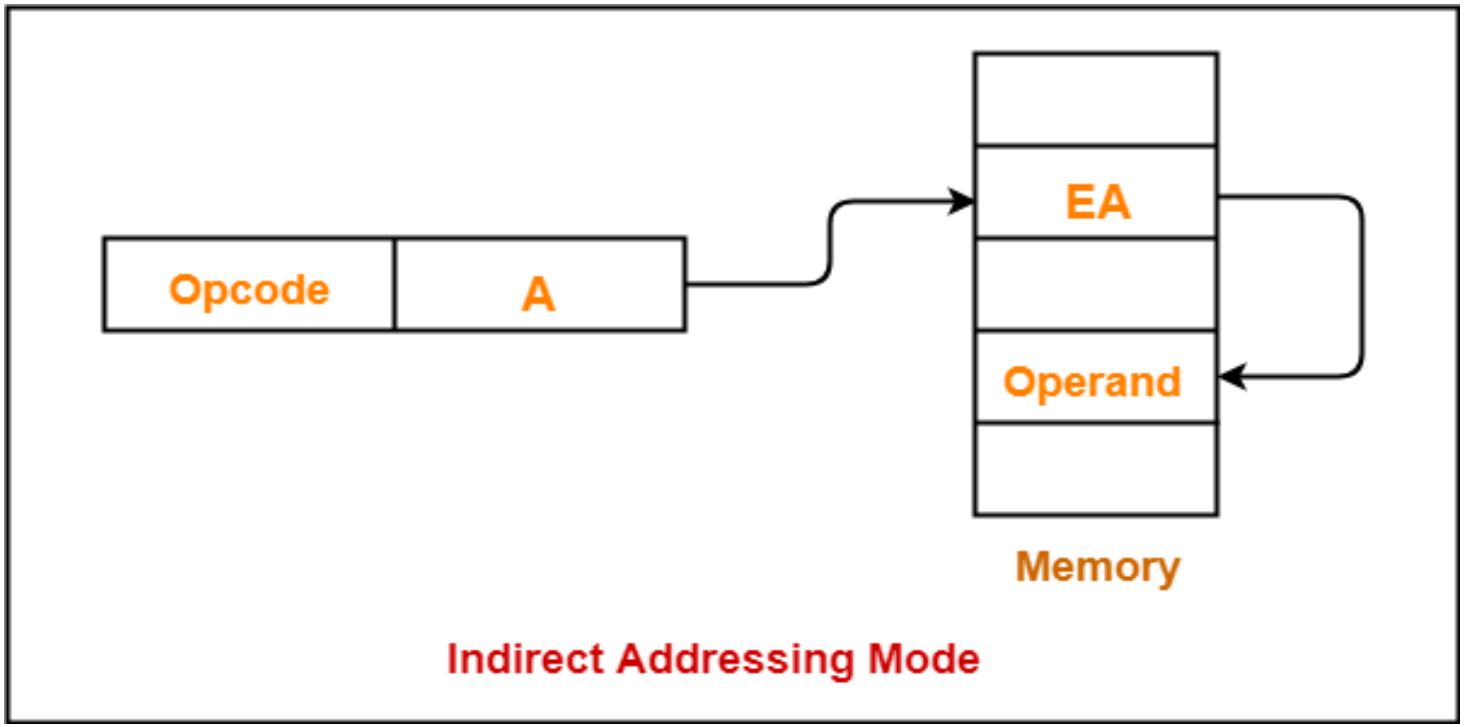
- Memory cell pointed to by address field contains the address of (pointer to) the operand

  Two references to memory are required to fetch the operand.
- Effective Address = [A]
  - Look in A, find address (A) and look there for operand

E.g. ADD (A)

  - Add contents of cell pointed to by contents of A to the accumulator

Indirect Addressing



Register Direct Addressing

In this addressing mode,

- The operand is contained in a register set.
- The address field of the instruction refers to a CPU register that contains the operand.
- No memory access
- Very fast execution
- Very limited address space
- Limited number of registers
- Very small address field needed
  - Shorter instructions
  - Faster instruction fetch

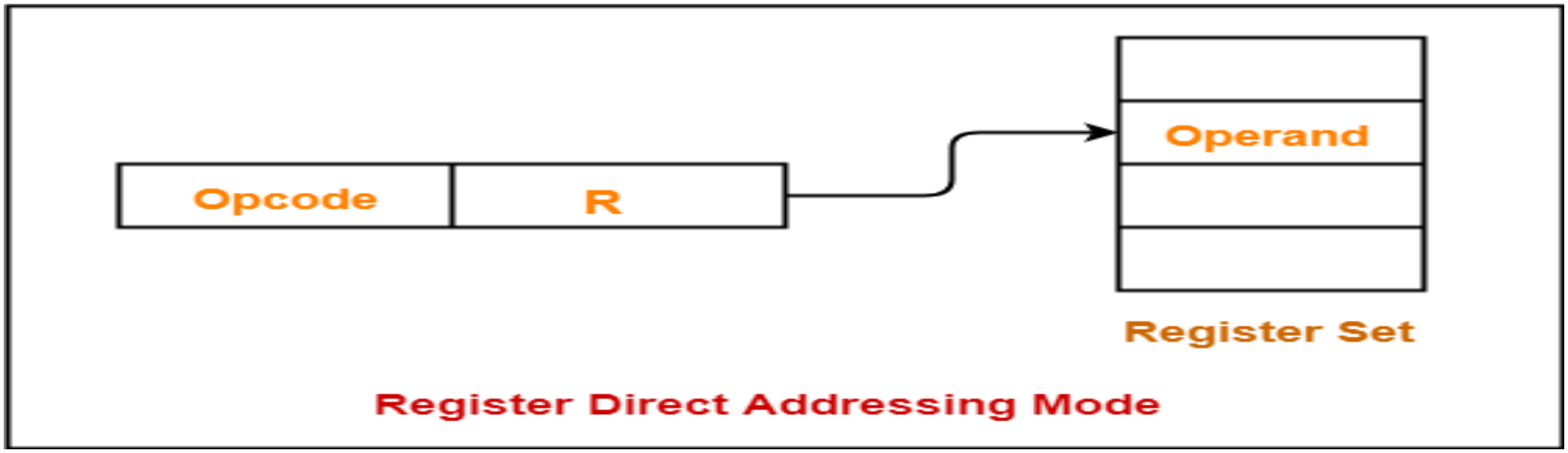
Register Direct Addressing

Eg:

ADD R will increment the value stored in the accumulator by the content of register R.

$$AC \leftarrow AC + [R]$$

- This addressing mode is similar to direct addressing mode.
- The only difference is address field of the instruction refers to a CPU register instead of main memory.

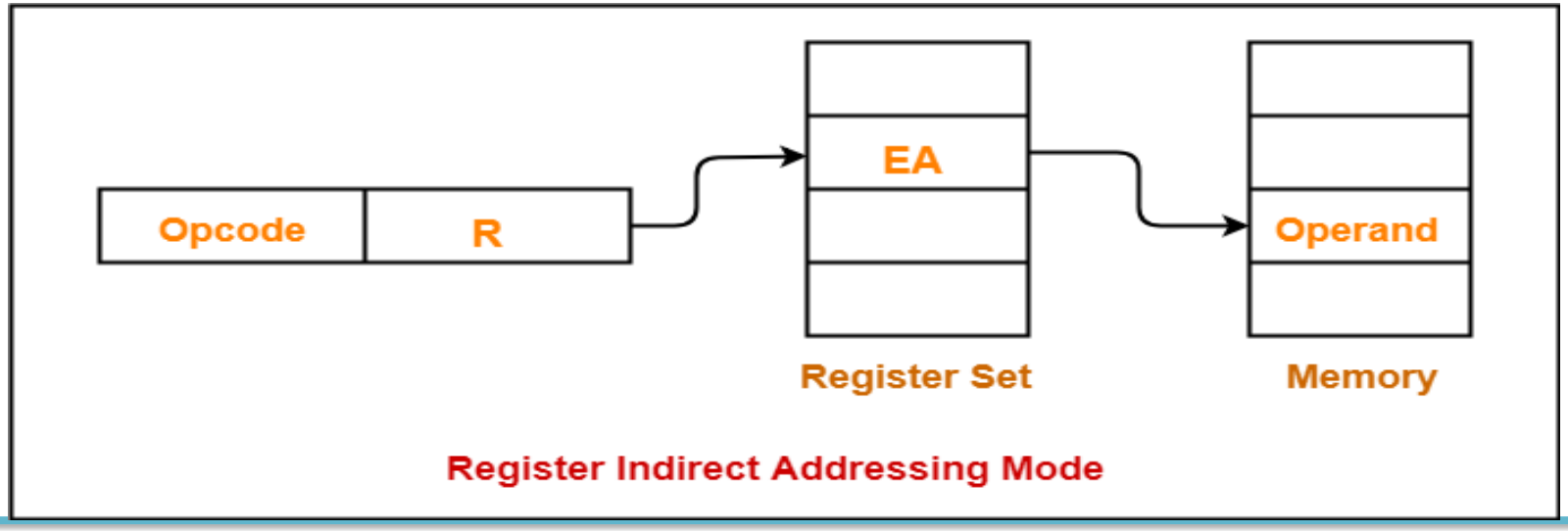


Register Indirect Addressing

- The address field of the instruction refers to a CPU register that contains the effective address of the operand.
- Only one reference to memory is required to fetch the operand

Eg: ADD R will increment the value stored in the accumulator by the content of memory location specified in register R.

$$AC \leftarrow AC + [[R]]$$



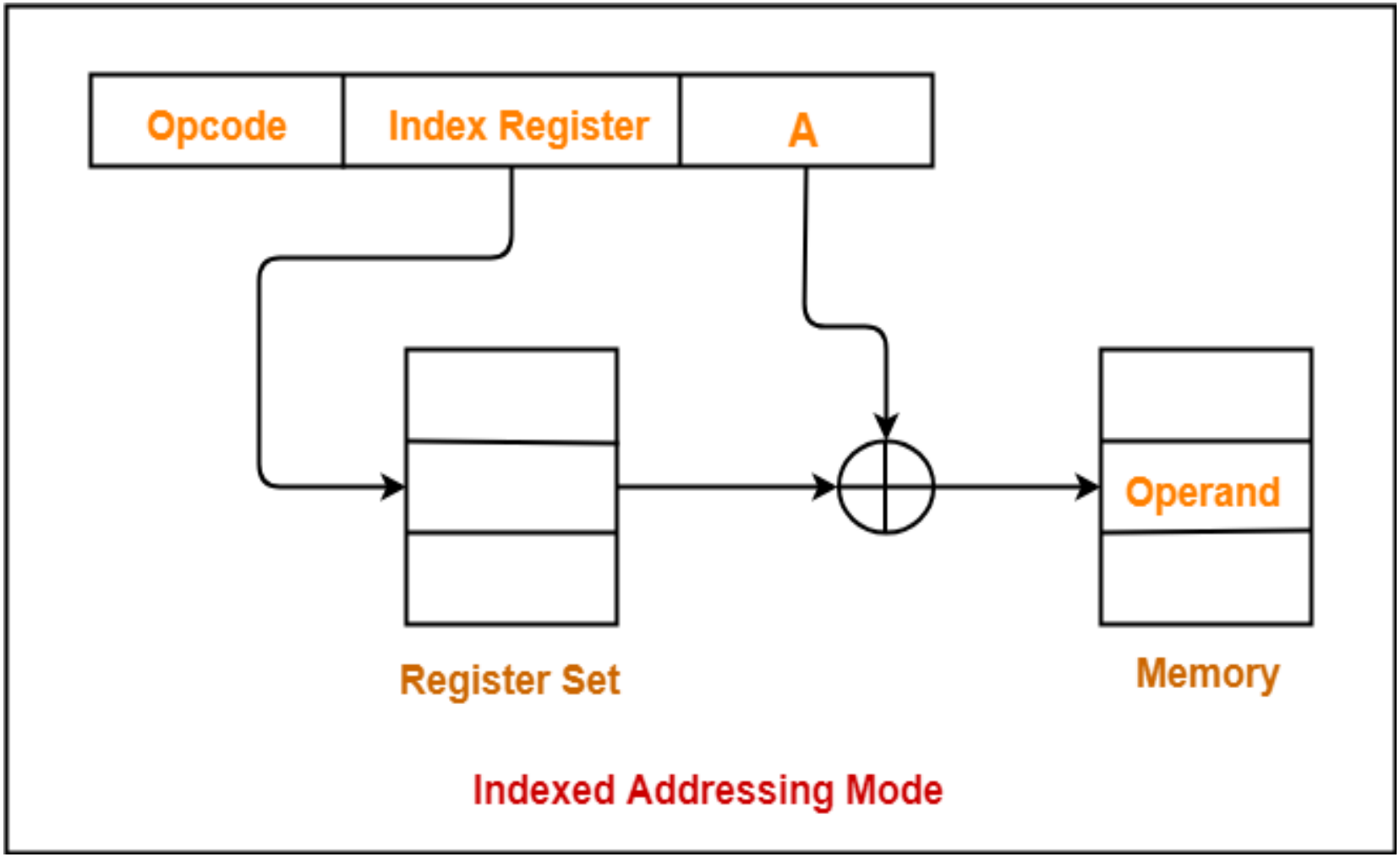
Indexed Addressing

In this addressing mode,

- Effective address of the operand is obtained by adding the content of index register with the address part of the instruction.

- **Effective Address = Content of Index Register + Address part of the instruction**

Indexed Addressing



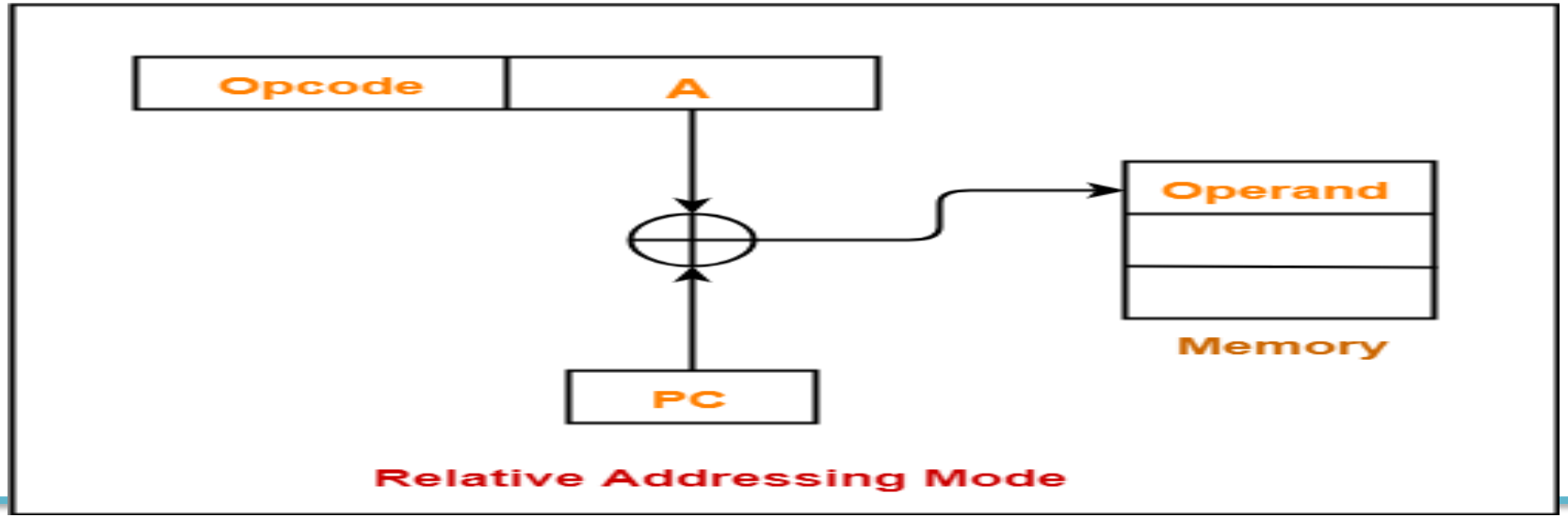
Relative Addressing

- A version of displacement addressing

In this addressing mode,

- Effective address of the operand is obtained by adding the content of program counter with the address part of the instruction.

Effective Address = Content of Program Counter + Address part of the instruction



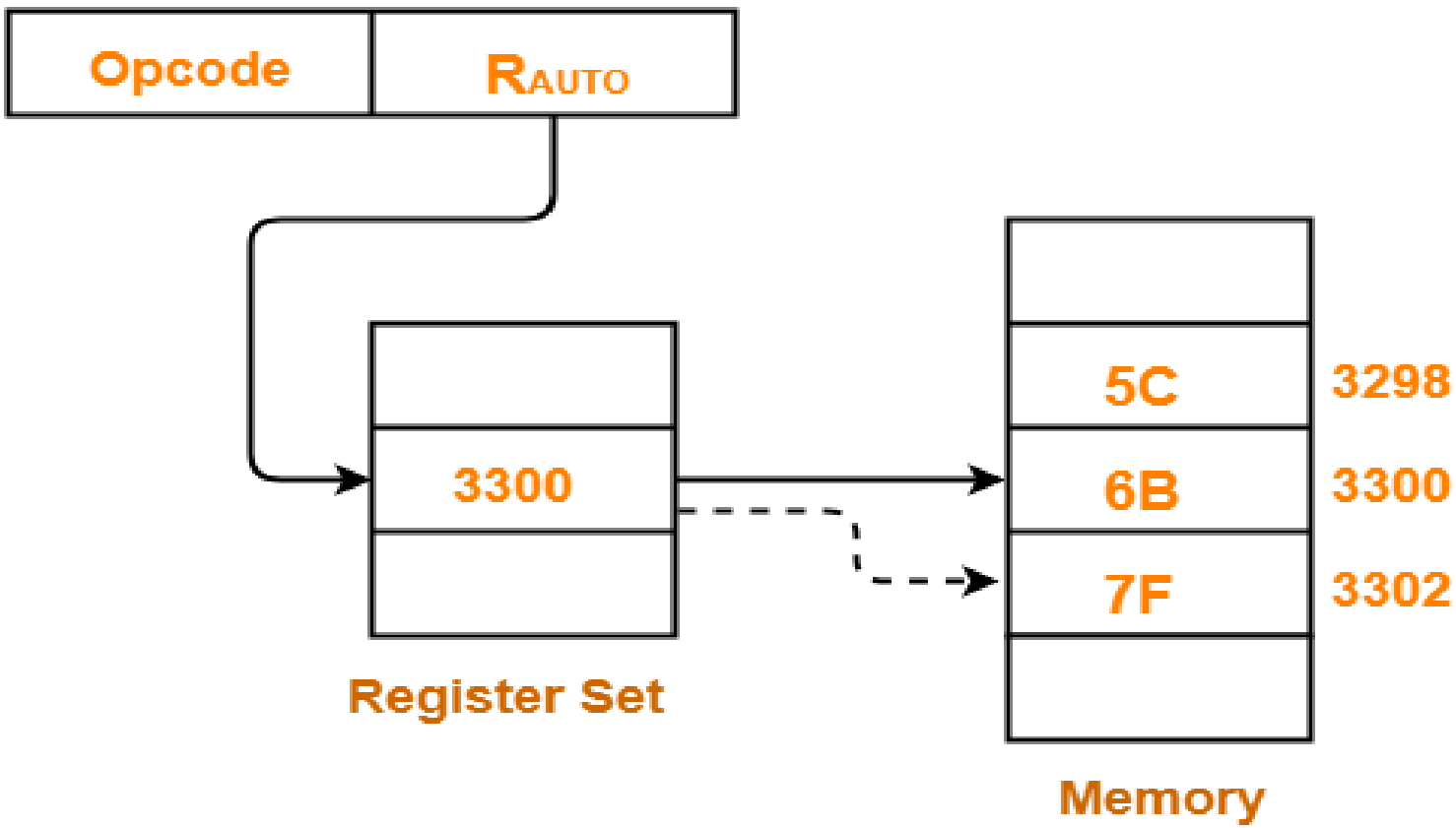
Auto increment mode

- A special case of Register Indirect Addressing Mode where,

Effective Address of the Operand = Content of Register

In this addressing mode,

- After accessing the operand, the content of the register is automatically incremented by step size ‘d’.
- Step size ‘d’ depends on the size of operand accessed.
- Only one reference to memory is required to fetch the operand.



Auto-Increment Addressing Mode

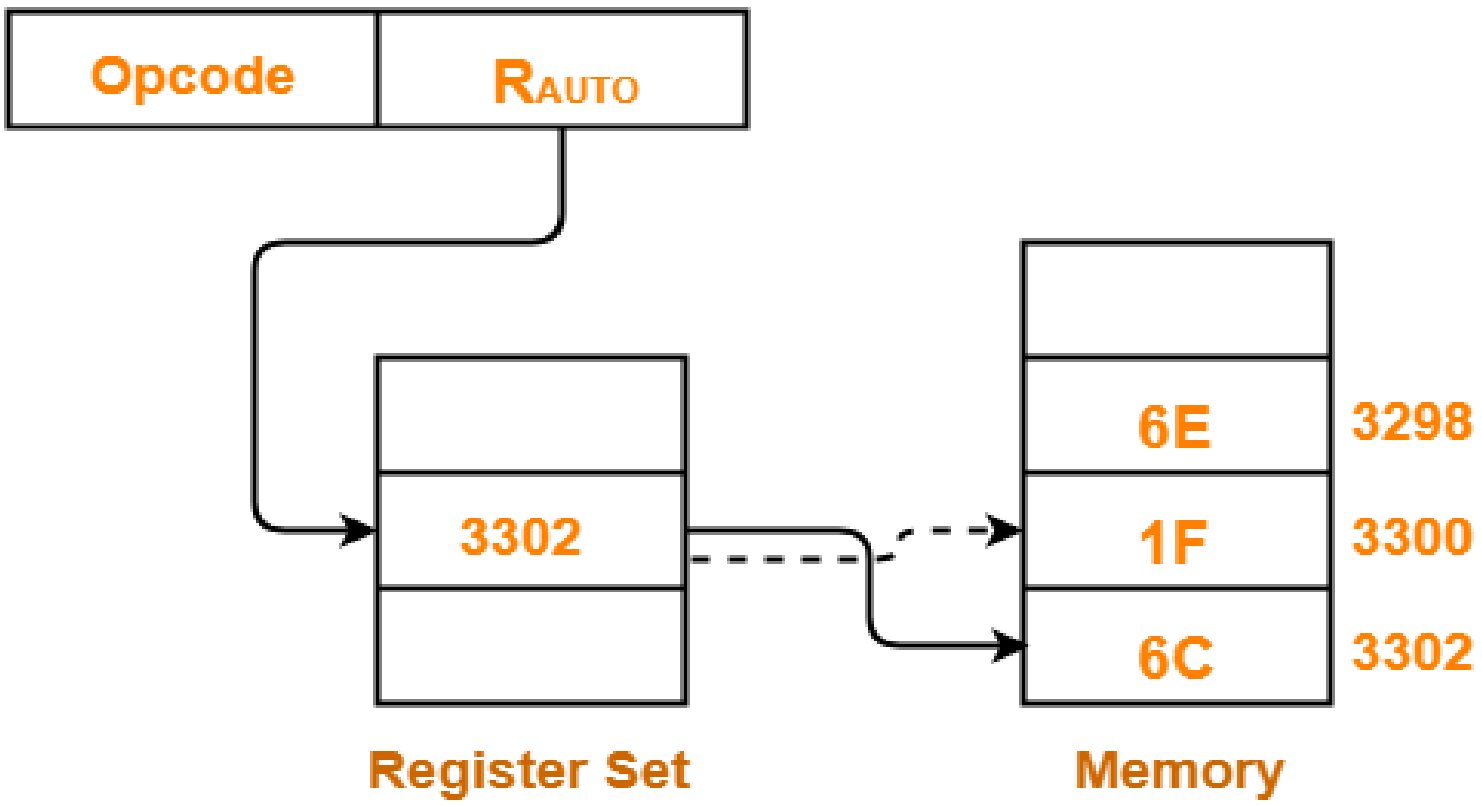
Auto decrement mode

- A special case of Register Indirect Addressing Mode where

Effective Address of the Operand = Content of Register – Step Size

In this addressing mode

- First, the content of the register is decremented by step size ‘d’.
- Step size ‘d’ depends on the size of operand accessed.
- After decrementing, the operand is read.
- Only one reference to memory is required to fetch the operand.



Auto-Decrement Addressing Mode

Instruction Binary Structures – 8086

- Assembly Language to Machine Instructions

Opcode	D	W	MOD	REG	R/M
6	1	1	2	3	2

- An instruction can be coded with 1 to 6 bytes
- Byte 1 contains three kinds of information
  - Opcode field is 6-bits specifies the operation such as add, subtract or move
  - Register Direction Bit (D-bit)
    - Tells the register operand in REG field in byte 2 is source or destination operand
      - 1: Data flow to the REG field from R/M
      - 0: Data flow from the REG to the R/M

Assembly Language to Machine Instructions

- Data Size Bit (W bit)
  - Specifies whether the operation will be performed on 8-bit or 16-bit data
    - 0: 8 – bits
    - 1: 16-bits

- Byte 2 has Two Fields
  - Mode field (MOD) – 2 bits
  - Register field (REG) – 3 bits
  - Register/ memory field (R/M field) – 2 bits
- REG field is used to identify the register for the first operand

Assembly Language to Machine Instructions

REG	W = 0	W = 1
000	AL	AX
001	CL	CX
010	DL	DX
011	BL	BX
100	AH	SP
101	CH	BP
110	DH	SI
111	BH	DI

MOD	Description
00	Memory Mode, No displacement follows*
01	Memory Mode, 8-bit displacement follows
10	Memory Mode, 16-bit displacement follows
11	Register Mode (no displacement)

* Except When R/M = 100, then 16-bit displacement follows

2-bit MOD Field and 3-bit R/M field together specify the second operand

Assembly Language to Machine Instructions

MOD = 11			Effective Address Calculation			
R/M	W=0	W=1	R/M	MOD=00	MOD=01	MOD=10
000	AL	AX	000	[BX] + [SI]	[BX] + [SI] + D8	[BX] + [SI] + D16
001	CL	CX	001	[BX] + [DI]	[BX] + [DI] + D8	[BX] + [DI] + D16
010	DL	DX	010	[BP] + [SI]	[BP] + [SI] + D8	[BP] + [SI] + D16
011	BL	BX	011	[BP] + [DI]	[BP] + [DI] + D8	[BP] + [DI] + D16
100	AH	SP	100	[SI]	[SI] + D8	[SI] + D16
101	CH	BP	101	[DI]	[DI] + D8	[DI] + D16
110	DH	SI	110	DIRECT ADDRESS	[BP] + D8	[BP] + D16
111	BH	DI	111	[BX]	[BX] + D8	[BX] + D16

Example

- MOV BL, AL
- Opcode MOV = 100 010
- AL – D=0; W bit = 0 (8-bits)
- MOD = 11 (Register Mode)
- REG = 000 (code for AL)
- R/M = 011 (Code for BL)

Opcode 100 010	D 0	W 0	MOD 11	REG 000	R/M 011

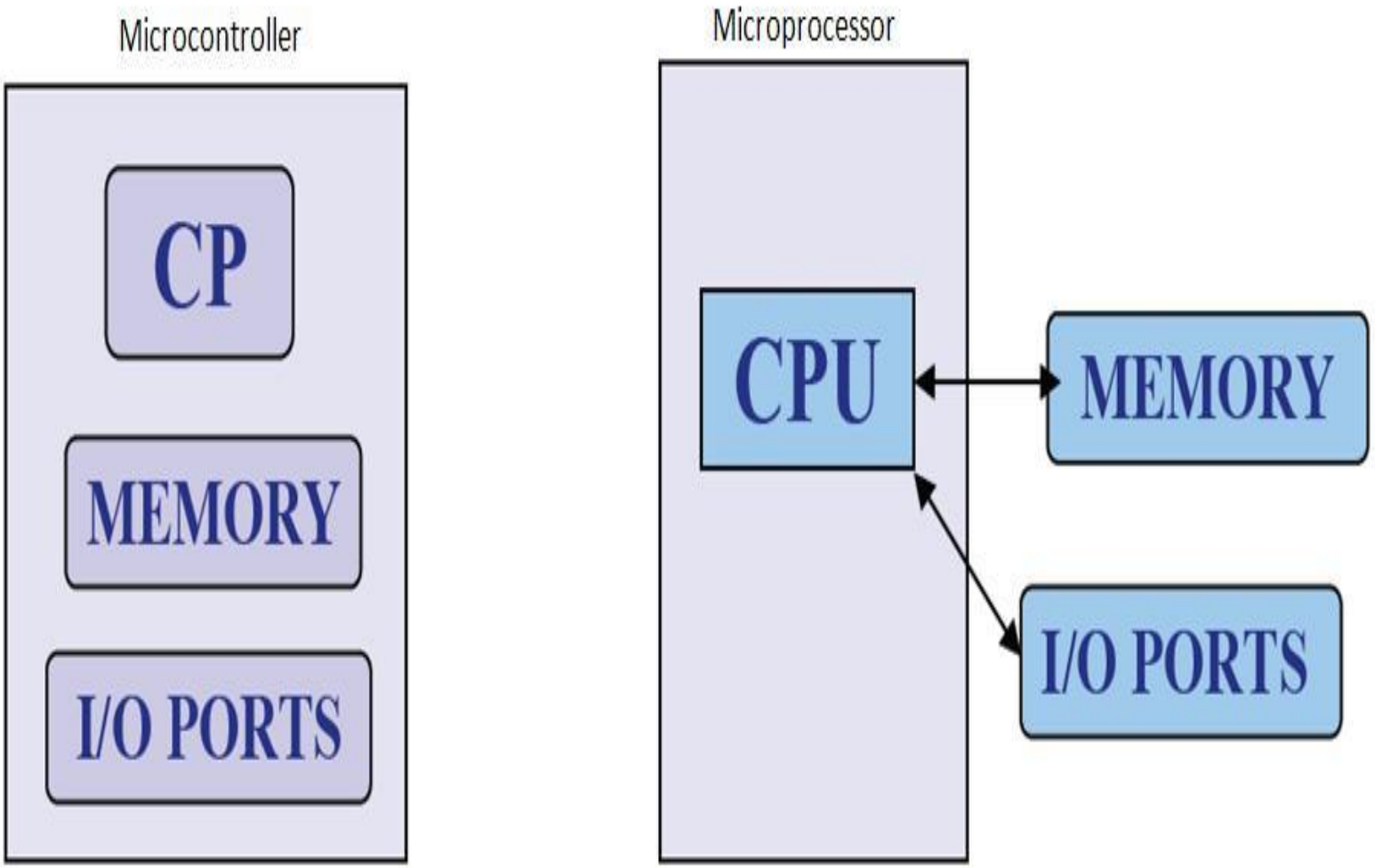
- MOV BL, AL = 10001000 11000011 = 88, C3H
- ADD AX, [SI] = 00000011 00000100 = 03, 04H
- ADD [BX][DI]+1234H, AX = 00000001 10000001 = 01, 81, 43, 12H

Introduction to Microprocessors

Microprocessors

- *Microprocessor* : is a CPU on a single chip.
- *Microcontroller*: If a microprocessor, its associated support circuitry, memory and peripheral I/O components are implemented on a single chip, it is a *microcontroller*.

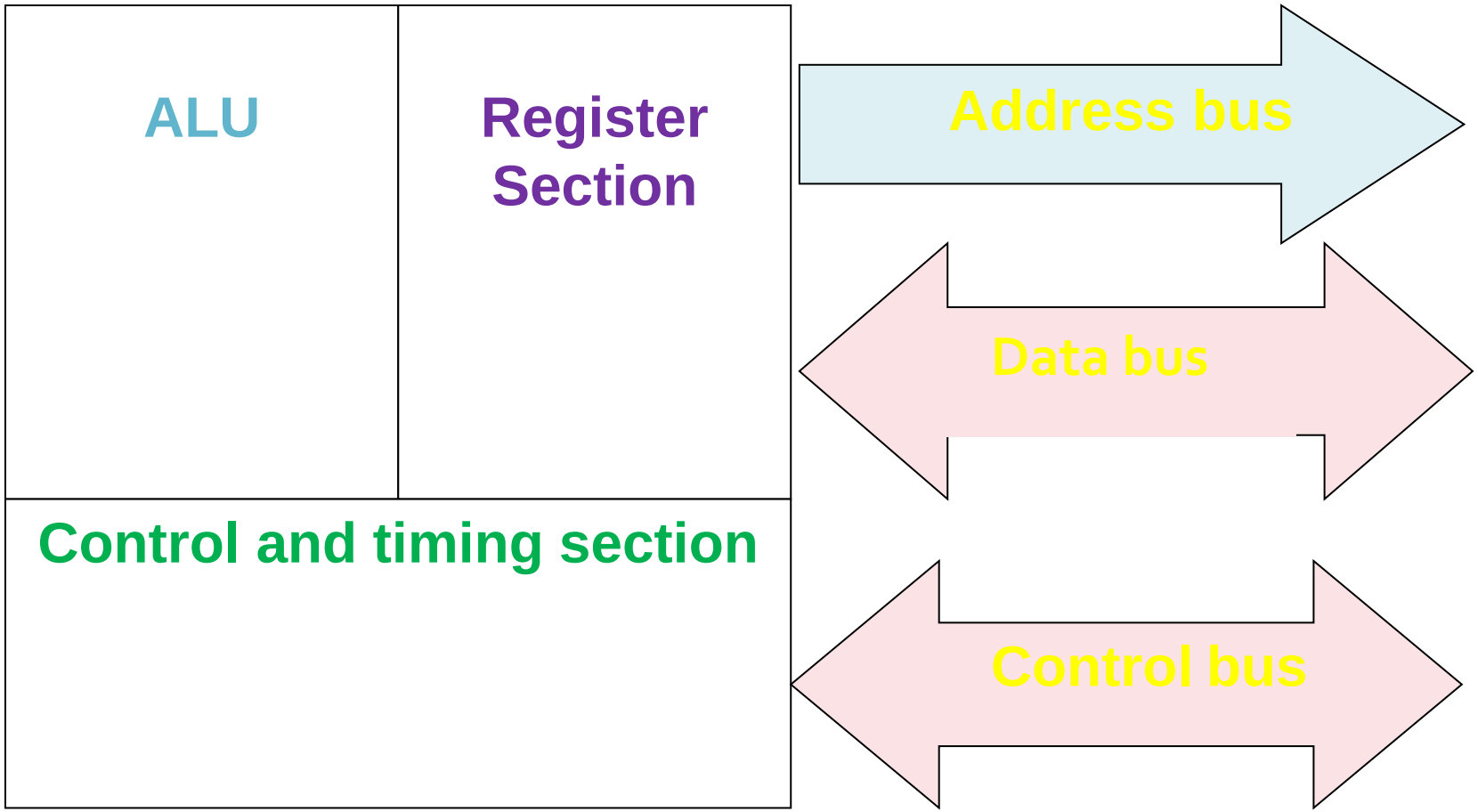
Diff. b/w Microprocessor & Microcontroller



Diff. b/w MP and MC

Sl No	Microprocessor	Microcontroller
1	CPU is stand-alone, RAM, ROM, I/O, timer are separate	CP,RAM,ROM,I/O and timer are all on single chip
2	Designer can decide on the amount of ROM,RAM and I/O ports	Fix amount of on-chip ROM, RAM, I/O ports
3	Expansive	Not Expansive
4	General purpose	Single purpose
5	Microprocessor based system design is complex and expensive	Microcontroller based system design is rather simple and cost effective
6	The instruction set of microprocessor is complex with large number of instructions.	The instruction set of a Microcontroller is very simple with the less number of instructions.

Block diagram of a Microprocessor



Functionalities of Microprocessors

- Microprocessor performs three main tasks:
  - data transfer between itself and the memory or I/O systems
  - simple arithmetic and logic operations
  - program flow via simple decisions

Microprocessor types

- Microprocessors can be characterized based on
  - the word size
    - 8 bit, 16 bit, 32 bit, etc. processors
  - Instruction set structure
    - RISC (Reduced Instruction Set Computer), CISC (Complex Instruction Set Computer)
  - Functions
    - General purpose, special purpose such image processing, floating point calculations
  - And more …

Evolution of Microprocessors

- The first **microprocessor** was **introduced** in 1971 by Intel Corp.
- It was named Intel 4004 as it was a 4-bit processor.

Categories according to the generations or size

First Generation (4 - bit Microprocessors)

- could perform simple arithmetic such as addition, subtraction, and logical operations like Boolean OR and Boolean AND.
- had a control unit capable of performing control functions like
  - fetching an instruction from storage memory,
  - decoding it, and then
  - generating control pulses to execute it.

Evolution of Microprocessors

- **Second Generation (8 - bit Microprocessor)**
  - The second generation microprocessors were introduced in 1973 again by Intel.
  - the first 8 - bit microprocessor which could perform arithmetic and logic operations on 8-bit words.
- **Third Generation (16 - bit Microprocessor)**
  - introduced in 1978
  - represented by **Intel's 8086, Zilog Z800 and 80286**,
  - 16 - bit processors with a performance like minicomputers.

Evolution of Microprocessors

- **Fourth Generation (32 - bit Microprocessors)**
  - Several different companies introduced the 32-bit microprocessors
  - the most popular one is the **Intel 80386**
- **Fifth Generation (64 - bit Microprocessors)**
  - Introduced in 1995
  - After 80856, Intel came out with a new processor namely Pentium processor followed by **Pentium Pro CPU**
  - allows multiple CPUs in a single system to achieve multiprocessing.
  - Other improved 64-bit processors are **Celeron, Dual, Quad, Octa Core processors**.

Typical Microprocessors

Most commonly used

- 68K
  - Motorola
- x86
  - Intel
- IA-64
  - Intel
- MIPS
  - Microprocessor without interlocked pipeline stages
- ARM
  - Advanced RISC Machine
- PowerPC
  - Apple-IBM-Motorola alliance
- Atmel AVR

8086 Microprocessor

- designed by Intel in 1976; 16-bit Microprocessor having 20 address lines and 16 data lines
- provides up to 1MB storage
- consists of powerful instruction set, which provides operations like multiplication and division easily.
- supports two modes of operation
  - Maximum mode :

    suitable for system having multiple processors
  - Minimum mode :

    suitable for system having a single processor.

Features of 8086

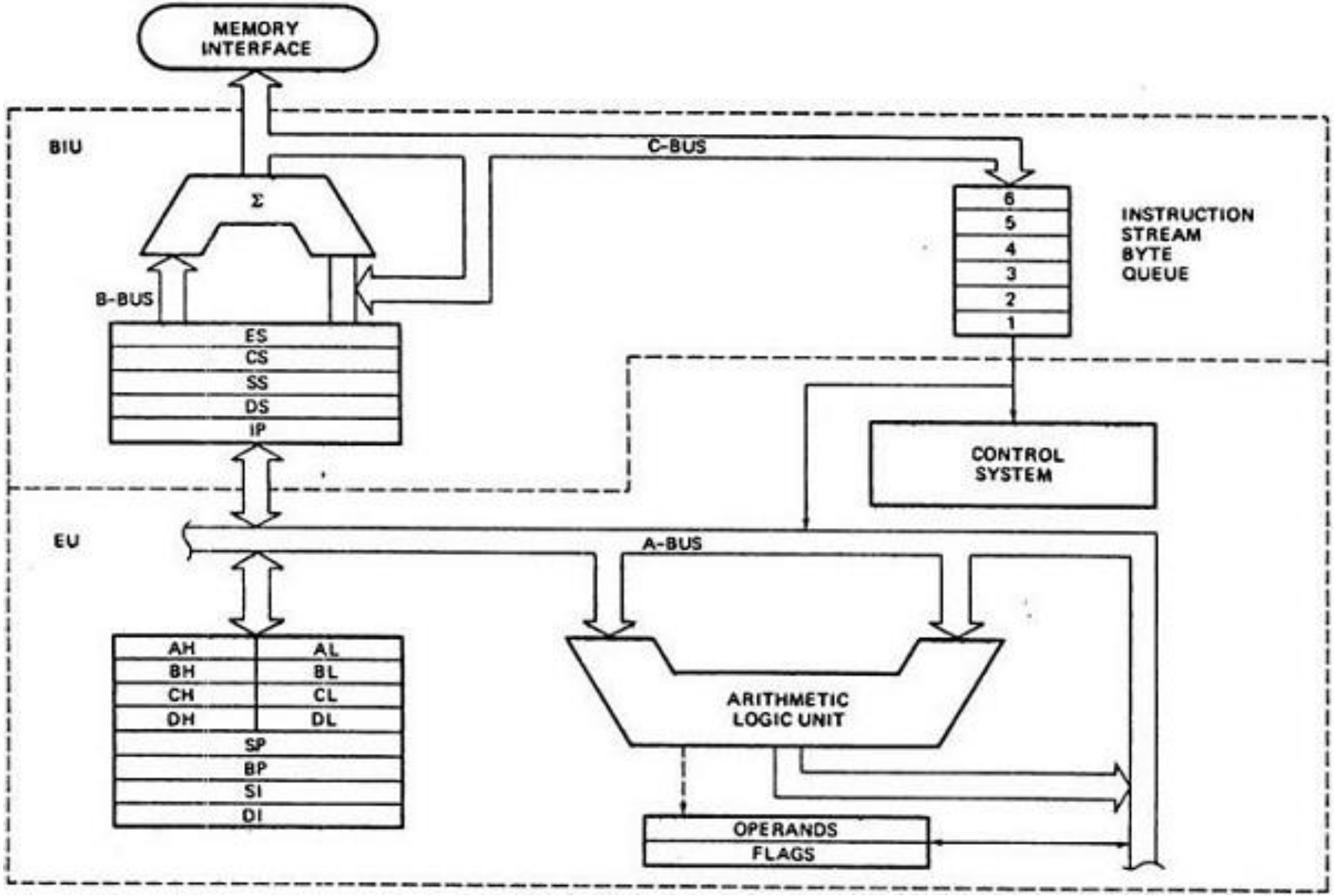
- Has an instruction queue, which can store six instruction bytes
- First 16-bit processor having
  - 16-bit ALU
  - 16-bit registers
  - internal data bus
  - 16-bit external data bus
- uses two stages of pipelining:

  1. Fetch Stage and 2. Execute Stage which improves performance.

  Fetch stage: Can pre-fetch up to 6 bytes of instructions and stores them in the queue.

  Execute stage : executes these instructions.

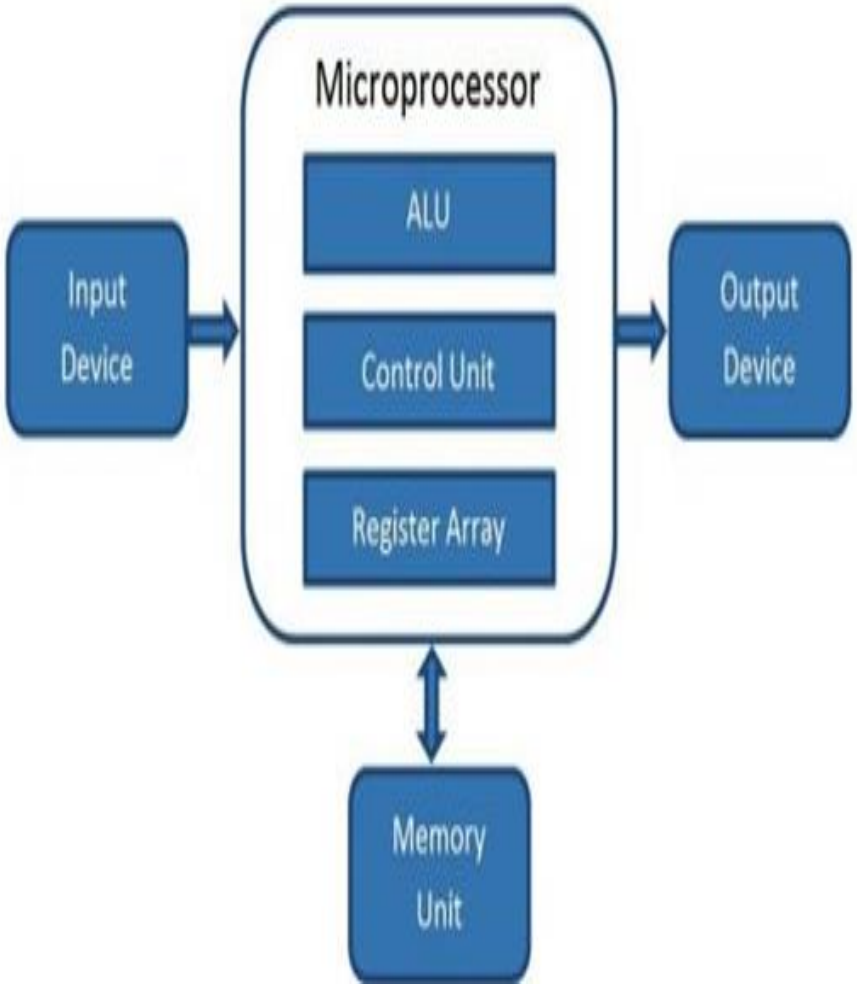
Architecture of 8086



Microcomputer

- A digital computer with one microprocessor which acts as a CPU
- A complete computer on a small scale, designed for use by one person at a time
- called a personal computer (PC)
- a device based on a single-chip microprocessor
- includes laptops and desktops

Block Diagram



Performance

Measure of performance of a computer is how quickly it can execute programs.

Speed with which a computer executed programs is affected by :

- Design of its hardware
- Machine language instructions
- Compiler that translates high-level language into machine language

For best performance, coordination is required b/w:

- Compiler design
- Machine instruction set

Parameters affecting the processor time

Processor clock

- Processor circuits are controlled by a timing signal called clock.
- Clock defines regular time intervals, called clock cycles.
- To execute a machine instruction, processor divides the action to be performed in steps.
- Each step completed in one clock cycle.
  - P= length of one clock cycle (Affects processor performance)
  - Clock rate, R= 1/P (cycles per second or hertz)
- Today's processors have few hundred million to billion cycles per second.

Basic performance equation

- Compiler generates machine language program corresponding to source program.
- Complete execution requires N machine language instructions.

N- actual number of instruction executions

- Average number of basic steps to execute one machine instructions is, S in one clock cycle.
- If clock rate is R cycles per second,

Processor time required to execute a program that is in high-level language is

$$T=(N \times S)/R$$

- To achieve high performance, reduce T by reducing N and S and increasing R.

Pipelining and superscalar operation

- Performance can be improved by overlapping the execution of successive instructions using pipelining
- Consider the instruction

Add R1,R2,R3

Adds the contents of R1 and R2 and result is placed in R3

- The processor can read the next instruction while addition being performed.
- Higher degree of concurrency can be achieved if multiple instruction pipelines are implemented.

Superscalar execution:

- Multiple functional units are used creating parallel paths for different instructions

Clock Rate

- Possibilities for increasing the clock rate, R

1. Improving the integrated-circuit (IC) technology
   - to make logic circuits faster by reducing the time.
   - The above will allow the clock period, P to reduce and clock rate, R is increased.

2. Reducing the amount of processing done in one basic step reduces clock period, P.

Instruction Set: CISC and RISC

- Simple instructions require a small number of basic steps to execute.
  - Here, large number of instructions are needed
  - N increases and S decrease.
- Complex instructions involve large number of steps.
  - Fewer instructions are needed.
  - Lower value of N and S increases.
- Design of the instruction set of a processor and the options include:
  - Reduced Instruction Set Computers (RISC) - Processors with simple instructions.
  - Complex Instruction Set Computers (CISC) - Processors with complex instructions.

Complier

- Compiler translates high-level language program into a sequence of machine instructions.
- To reduce N, suitable machine instruction set and compiler
- *Optimized compiler* reduces the product N X S,

  which is total number of clock cycles needed to execute a program.
- Compiler may rearrange program instructions to achieve better performance.
- High-quality compiler must be closely linked to the processor architecture.
- Objective is to reduce the total number of clock cycles needed to perform a required programming task

Performance Measurement

- The performance measure is the time taken by the computer to execute a given bench-mark.
- A non-profit organization called SPEC- System Performance Evaluation Corporation selects and publishes bench-marks.
- The program selected range from game playing, compiler, and data base applications to numerically intensive programs in astrophysics and quantum chemistry.
- The test is repeated for all the programs in the SPEC suite and geometric mean is computed.

$$SPEC\ Rating = \frac{Running\ time\ on\ the\ reference\ computer}{Running\ time\ on\ the\ computer\ under\ test}$$

Performance Measurement

Final SPEC Rating

$$SPEC\ Rating = \left[\left[\prod_{i-1}^{n} SPECi\right]^{1/n}\right]$$

END